Universität Leipzig

Wirtschaftswissenschaftliche Fakultät

Institut für Wirtschaftsinformatik

Professor Dr. Ulrich Eisenecker

Janik Eriksson

Thema

Effiziente Generierung kombinatorischer Strukturen ohne Speicherung aller Zustände in C++

Bachelorarbeit zur Erlangung des akademischen Grades

Bachelor of Science - Wirtschaftsinformatik

vorgelegt von: Hempel, Anton

Matrikelnummer: 3741358

E-Mail-Adresse: ny22efab@studserv.uni-leipzig.de

Anschrift: Mittelstr. 52, 04463 Großpösna OT Seifertshain

Abstract

Kombinatorische Überlegungen können im täglichen Leben an den unterschiedlichsten Stellen Anwendung finden. Diese Alltagsprobleme lassen sich mit den Methoden der Kombinatorik lösen. Häufig werden dafür die bekanntesten kombinatorischen Strukturen Permutation und Kombination verwendet, welche sich in mit und ohne Wiederholung weiter untergliedern lassen. Bei der Generierung kombinatorischer Strukturen wird bei den meisten Anwendungen auf iterative oder rekursive Ansätze vertraut. Dabei werden Zwischenergebnisse gespeichert oder als Argument übergeben, um auf Basis dieser das nächste Ergebnis zu berechnen. Unter der Annahme, dass diese benötigten Zwischenerbnisse gespeichert werden, führt dies zu einem hohen Speicherbedarf. Ranking- und Unranking-Algorithmen bieten eine speicheroptimierte Lösung, indem sie eine Bijektion zwischen den Elementen und einem dazugehörigen Rang bilden. In der Arbeit soll untersucht werden, inwieweit sich kombinatorische Strukturen ohne das Speichern von Zuständen oder Zwischenergebnissen generieren lassen. Als Ergebnis der Arbeit wird eine praktische Implementierung in der Programmiersprache C++ bereitgestellt, welche mittels von Unranking-Algorithmen die technische Machbarkeit beweist. Anschließend werden Zeit- und Speicherbenchmarks durchgeführt, um den Ansatz mit Ergebnissen anderer Ansätze zu vergleichen und diesen einzuordnen.

Inhaltsverzeichnis

Inhaltsverzeichnis I
Abbildungsverzeichnis III
Tabellenverzeichnis IV
Liste der Algorithmen V
Abkürzungsverzeichnis VI
1 Einleitung 1
1.1 Motivation und Problemstellung 1
1.2 Zielstellung 2
1.3 Forschungsmethode 2
2 Permutation und Kombination 3
2.1 Mengen und Folgen 3
2.2 Permutation ohne Wiederholung 3
2.3 Permutation mit Wiederholung 4
2.4 Kombination ohne Wiederholung 5
2.5 Kombination mit Wiederholung 6
3 Urnenmodell und Anwendungsfälle 8
3.1 Grundbegriffe im Urnenmodell 8
3.2 Auswahl mit Beachtung der Reihenfolge und ohne Wiederholungsmöglichkeit 9
3.3 Auswahl mit Beachtung der Reihenfolge und Wiederholungsmöglichkeit 9
3.4 Auswahl ohne Beachtung der Reihenfolge und ohne Wiederholungsmöglichkeit 10
3.5 Auswahl ohne Beachtung der Reihenfolge und Wiederholungsmöglichkeit 10
3.6 Zusammenfassung des Urnenmodells 12
4 Kombinatorische Unranking-Algorithmen 12
4.1 Ranking- und Unranking-Algorithmen 12
4.2 Anwendungsfälle für Ranking- und Unranking-Algorithmen 13
4.3 Lexikografische Ordnung 14
5 Die Standard Template Library 14
5.1 Komponenten der STL 14
5.2 STL-Iteratoren 15
6 Unranking-Algorithmen für das Urnenmodell 16
6.1 Unranking für Permutation ohne Wiederholung 16
6.2 Unranking für Permutation mit Wiederholung 20
6.3 Unranking für Kombination ohne Wiederholung 22
6.4 Unranking für Kombination mit Wiederholung 25
7 Konzeption und Implementierung des Prototyps 27

7.1 Prototyp des Urnenmodells in C++ 27
7.2 Erweiterung des Prototyps um einen C++-Iterator 29
7.3 Prototyp Anwendungsbeispiel 33
8 Benchmarks 34
8.1 Zeitbenchmarks 35
8.2 Speicherbenchmarks 39
8.3 Auswertung der Ansätze 42
9 Fazit 43
9.1 Bewertung der Ergebnisse der Arbeit 43
9.2 Ausblick 44
A Quellcode VI
B Benchmark Ergebnisse VI
Literaturverzeichnis X
10 Ehrenwörtliche Erklärung XII

Abbildungsverzeichnis

Abbildung 1: Übersicht Urnenmodell (in Anlehnung an [Berger 2023, 47]) 12
Abbildung 2: Hierarchie der Iterator Kategorien (in Anlehnung an [Josuttis 2012, 434]) .. 15
Abbildung 3: UML-Klassendiagramm des Urnenmodell-Prototyps 28
Abbildung 4: Beispiel Anwendung UrnOR 29
Abbildung 5: UML-Klassendiagramm des Urnenmodell-Prototyps mit Iterator 30
Abbildung 6: UrnIterator Implementation des Präfix-Increment Operators 31
Abbildung 7: Anwendungsbeispiel des Indexoperators 32
Abbildung 8: Anwendungsbeispiel der Ranges Filterfunktion 34
Abbildung 9: Anwendungsbeispiel 2 der Ranges Filterfunktion 34
Abbildung 10: Anwendungsbeispiel des naiven Ansatzes für Urn 35
Abbildung 11: Benchmark: Speicherverbrauch des naiven Ansatzes NaiveUrn 40
Abbildung 12: Benchmark: Speicherverbrauch des Prototyps Urn 41
Abbildung 13:Benchmark: Speicherverbrauch Discreture combinations 42

Tabellenverzeichnis

Tabelle 1: Aufgaben/Fähigkeiten der Iterator-Kategorien (in Anlehnung an [Josuttis 2012, 434]) 15
Tabelle 2: Vergleich der Ziehungen für K(4,3) und K*(2,3) 25
Tabelle 3: Benchmark: Volle Iteration k-Permutation mit Wiederholung 36
Tabelle 4: Benchmark: Spezifische Ziehung k-Permutation mit Wiederholung 38
Tabelle 5: Benchmark: Volle Iteration k-Permutation ohne Wiederholung..... VI
Tabelle 6: Benchmark: Spezifische Ziehung k-Permutation ohne Wiederholung.....VII
Tabelle 7: Benchmark: Volle Iteration k-Kombination mit Wiederholung..... VIII
Tabelle 8: Benchmark: Spezifische Ziehung k-Kombination mit Wiederholung..... VIII
Tabelle 9: Benchmark: Volle Iteration k-Kombination ohne Wiederholung IX
Tabelle 10: Benchmark: Spezifische Ziehung k-Kombination ohne Wiederholung: IX

Liste der Algorithmen

Algorithmus 1: Unranking Permutation 20
Algorithmus 2: Unranking k-Permutation ohne Wiederholung in C++ 20
Algorithmus 3: Basis-Expansion (in Anlehnung an [Rosen 2012, 249]) 21
Algorithmus 4: Unranking k-Permutation mit Wiederholung in C++ (in Anlehnung an [Rosen 2012, 249]) 22
Algorithmus 5: Unranking k-Kombinationen ohne Wiederholung (in Anlehnung an [Genitrini/Pépin 2021, 7]) 23
Algorithmus 6: Rangkonvertierung (in Anlehnung an [Genitrini/Pépin 2021, 7]) 23
Algorithmus 7: Unranking k-Kombinationen ohne Wiederholung in C++ (in Anlehnung an [Genitrini/Pépin 2021, 20]) 24
Algorithmus 8: Unranking k-Kombinationen mit Wiederholung 26
Algorithmus 9: Unranking k-Kombinationen mit Wiederholung in C++ 26

Abkürzungsverzeichnis

$\boldsymbol{P(n,k)}$ *k*-Permutation ohne Wiederholung

$\boldsymbol{P^*(n,k)}$ *k*-Permutation mit Wiederholung

$\boldsymbol{K(n,k)}$ *k*-Kombination ohne Wiederholung

$\boldsymbol{K^*(n,k)}$ *k*-Kombination mit Wiederholung

STL Standard Template Library

KI Konfidenzintervall

1 Einleitung

1.1 Motivation und Problemstellung

Kombinatorische Überlegungen können im täglichen Leben an den unterschiedlichsten Stellen Anwendung finden. „Wie viele Möglichkeiten gibt es beim Lotto, sechs Zahlen aus 49 auszuwählen?“ „Wie viele verschiedene Möglichkeiten gibt es, Personen in einem Gruppenfoto aufzustellen?“ Auch in der Informatik gibt es zahlreiche Problemstellungen, für welche diese Verfahren verwendet werden. Eins der bekanntesten ist wohl: „Wie viele Passwörter einer bestimmten Länge können mit alphanumerischem Zeichen und Sonderzeichen generiert werden?“ All diese Fragen können mit Methoden der Kombinatorik beantwortet werden. Solche Alltagsprobleme lassen sich häufig mit den kombinatorischen Strukturen der Permutation oder Kombination lösen. Diese kombinatorischen Verfahren lassen sich jeweils in mit und ohne Wiederholung untergliedern (vgl. [Witt 2013, V-1]). Zusätzlich gibt es verschiedene Modelle wie z.B. das Urnenmodell, um ein besseres Verständnis für diese Strukturen zu bekommen (vgl. [Berger 2023, 30–39]).

Bei vielen Anwendungen werden kombinatorische Strukturen systematisch erzeugt oder durchlaufen. Typischerweise werden dafür iterative oder rekursive Ansätze verwendet, welche alle möglichen Zustände nacheinander erzeugen. Hier werden oft Zwischenergebnisse gespeichert oder vorherige Zustände als Argument übergeben, um auf Basis dieser den nächsten Zustand zu berechnen. Beispiele für die Verwendung von Zwischenergebnissen wären *std::next_permutation* aus der C++-Standardbibliothek (vgl. [cplusplus.com 2025]) oder der Ansatz von [Mifsud 1963]. Mit zunehmender Größe der kombinatorischen Struktur steigt die Anzahl der zu generierenden Zwischenergebnisse. Unter der Annahme, dass die benötigten Zwischenergebnisse gespeichert werden, führt dies zu einem hohen Speicherbedarf und einer ineffizienten Nutzung von Ressourcen. Insbesondere für Systeme mit begrenzten Speicherressourcen könnte das problematisch sein.

Eine speicheroptimierte Lösung könnten Ranking- und Unranking-Algorithmen bieten. Diese sind inverse Operationen zueinander und stellen eine Bijektion vom Element zu dem entsprechenden Rang (Index) her. Besonders relevant für die Arbeit sind Unranking-Algorithmen. Unranking-Algorithmen beschreiben die Operationen, welche nötig sind, um mit einem gegebenen Rang (Index) das dazugehörige Element zu finden (vgl. [Frank Ruskey 2003, 5–6]). Im Gegensatz zu den iterativen und rekursiven Methoden müssten hierbei nur die relevanten Indizes und nicht die Zwischenergebnisse selbst gespeichert werden.

1.2 Zielstellung

In der Arbeit soll untersucht werden, inwieweit sich kombinatorische Strukturen ohne das Speichern von Zuständen oder Zwischenergebnissen generieren lassen. Ebenfalls soll eine praktische Implementierung für die Programmiersprache C++ bereitgestellt werden. Daraus ergibt sich die Frage: Inwiefern können kombinatorische Strukturen durch C++-Iteratoren ohne unterliegenden Container umgesetzt werden? Weiterhin soll die Frage beantwortet werden, wie eine solche Implementierung gegenüber anderen kombinatorischen Implementierungen im Hinblick auf Laufzeit- und Speicherperformance einzuordnen ist. Zur Beantwortung der Fragen werden zuerst bestehende wissenschaftliche Ansätze für kombinatorische Unranking-Algorithmen untersucht und kritisch bewertet. Die ausgewählten Algorithmen werden in C++ implementiert und dienen für die Entwicklung eines Prototyps des Urnenmodells. Zusätzlich soll der Prototyp als C++-Iterator implementiert werden, um eine hohe praktische Anwendbarkeit zu gewährleisten. Anschließend werden mit dem Prototyp verschiedene Zeit- und Speicherbenchmarks durchgeführt, um diese mit Ergebnissen anderer Ansätze zu vergleichen.

1.3 Forschungsmethode

[Wilde/Hess 2006, 3] unterteilen das Forschungsgebiet der Wirtschaftsinformatik in zwei erkenntnistheoretische Positionen. Zum einen gibt es das konstruktionswissenschaftliche Paradigma („Design Science“), das sich auf das Schaffen und Evaluieren von IT-Modellen konzentriert. Zum anderen identifizieren sie das verhaltenswissenschaftliche Paradigma („Behavioral Science“), welches sich durch das Analysieren von Verhalten und der Auswirkung von bestehenden Informationssystemen auf Organisationen auszeichnet.

Im Rahmen der Bachelorarbeit finden beide Paradigmen ihre Anwendung. Am offensichtlichsten ist dabei der Einsatz des konstruktionswissenschaftlichen Paradigmas. Hier wird die Methode mit dem plastischsten Konstruktionsbezug, das Prototyping, verwendet. Beim Prototyping geht es darum, eine schnell verfügbare und lauffähige Version zu entwickeln und diese zu evaluieren (vgl. [Wilde/Hess 2006, 6]). Im Kontext der Bachelorarbeit wird ein Prototyp des Urnenmodells entwickelt und evaluiert. Anschließend wird das verhaltenswissenschaftliche Paradigma im Rahmen eines Laborexperiments angewandt. Ein Laborexperiment ist nach Wilde/Hess (vgl. [Wilde/Hess 2006, 9]) durch die Untersuchung von Kausalzusammenhängen in einer kontrollierten Umgebung definiert. Dabei wird eine Experimentvariable wiederholbar manipuliert (Stimulus) und die Auswirkung der Manipulation (Response) gemessen. In der Bachelorarbeit werden hierzu in

einer künstlich geschaffenen Umgebung verschiedene Zeit- und Speicherbenchmarks durchgeführt. Da die Untersuchung in einer künstlichen Umgebung erfolgt, entspricht dies einem Laborexperiment.

2 Permutation und Kombination

In diesem Kapitel werden die Grundlagen zweier kombinatorischer Strukturen, der Permutation und der Kombination, behandelt. Jede der beiden kombinatorischen Strukturen lässt sich jeweils weiter unterteilen, nämlich in solche mit Wiederholung und solche ohne Wiederholung. Diese Grundlagen sind notwendig, um im nächsten Kapitel das Urnenmodell zu verstehen.

2.1 Mengen und Folgen

Bei kombinatorischen Strukturen kann die Anordnung der Elemente eine Rolle spielen. Angenommen, es gibt eine endliche Menge *M* mit den Elementen $M = \{1, 2, 3\}$ so besteht die Möglichkeit, je nach kombinatorischer Struktur, zu sagen $\{1, 2, 3\} = \{2, 1, 3\}$ oder $\{1, 2, 3\} \neq \{2, 1, 3\}$. Um eine klare Trennung zu erzielen, definieren wir im Folgenden den zweiten Fall als eine Folge, sodass $\langle 1, 2, 1\rangle \neq \langle 2, 1, 1\rangle$.

Definition Menge: Eine Menge $M = \{x_1, x_2, \dots, x_k\}, k \geq 0, x_i \in M, 1 \leq i \leq k, |M| = n$ ist eine Anordnung von Elementen, bei der die Anordnung der Elemente irrelevant ist. Eine Menge mit $k = 0$ ist eine leere Menge {}. Um die Notation von Mengen mit vielen Elemente kurz zu halten, kann diese ebenfalls als $M = \{1, \dots, n\} = [1, n]$ geschrieben werden.

Definition Folge: Eine Folge $F = \langle x_1, x_2, \dots, x_k\rangle, k \geq 0, x_i \in F, 1 \leq i \leq k, |F| = n$. Ist eine Auswahl von Elementen, bei der die Anordnung der Elemente einen Unterschied darstellt. Eine Folge mit $k = 0$ ist eine leere Folge $\langle\ \rangle$(vgl. [Witt 2013, 3]).

2.2 Permutation ohne Wiederholung

Definition k-Permutation ohne Wiederholung: Sei $M = [1, n]$ und $k \in \mathbb{N}_0$ mit $0 \leq k \leq n$. Dann kann aus der Menge *M* eine Folge F $\langle x_1, x_2, \dots, x_k\rangle$ mit $x_i \in M$ gewählt werden, wobei $x_i \neq x_j$ für $i \neq j, 1 \leq i, j \leq k$. Diese Folge F entspricht einer *k*-Permutation ohne Wiederholung.

Eine *k*-Permutation ohne Wiederholung entspricht damit einer geordneten Ziehung von *k* Elementen aus der Menge *M*, ohne dass Elemente wiederholt werden dürfen.

Um die Anzahl der möglichen Ziehungen einer k-Permutation ohne Wiederholung anzugeben, nutzen wir die Kurzform $P(n,k)$. Zum Beispiel ergeben sich für die Menge $M = \{0,1,2\}$ und $P(3,2) = 6$ die Folgen: $\langle 0,1\rangle$, $\langle 0,2\rangle$, $\langle 1,0\rangle$, $\langle 1,2\rangle$, $\langle 2,0\rangle$ und $\langle 2,1\rangle$ (vgl. [Witt 2013, 4]).

Zur Berechnung von $P(n,k)$ kann folgende Formel verwendet werden, vorausgesetzt $0 \leq k \leq n$ gilt.

$$P(n,k) = \frac{n!}{(n-k)!}$$

So ergibt sich beispielsweise für $P(3,2)$:

$$P(3,2) = \frac{3!}{(3-1)!} = \frac{6}{1} = 6$$

Ein Beweis der Formel findet sich in [Witt 2013, 4]. In [Witt 2013] und [Berger 2023] wird die Formel zur Berechnung der Anzahl von k-Permutationen ohne Wiederholung zusätzlich über das fallende Faktorielle definiert. Der Vollständigkeit und Wiedererkennbarkeit halber sei hier auch die folgende äquivalente Formel erwähnt:

$$\begin{aligned} P(n,k) = n^{\underline{k}} &= (n-0)(n-1)(n-2)\dots\big(n-(k-1)\big) \\ &= n(n-1)(n-2)\dots(n-k+1) \end{aligned} \tag{2.1}$$

Das fallende Faktorielle von n wird als „n hoch k fallend“ gesprochen (vgl. [Witt 2013, 4–5]). Es ist äquivalent zu:

$$P(n,k) = n^{\underline{k}} = \frac{n!}{(n-k)!}$$

2.3 Permutation mit Wiederholung

Wenn Wiederholungen in einer k-Permutation zulässig sind, wird sie wie folgt definiert.

Definition k-Permutation mit Wiederholung: Sei M eine Menge mit n Elementen mit $k \in \mathbb{N}_0$. Dann kann aus der Menge M eine Folge F $\langle x_1, x_2, \dots, x_k\rangle$ mit $x_i \in M, 1 \leq i \leq k$ gewählt werden. Diese Folge F definieren wir als k-Permutation mit Wiederholung.

Eine k-Permutation mit Wiederholung entspricht somit einer geordneten Auswahl von k Elementen aus der Menge M, wobei Elemente mehrfach vorkommen können.

Um die Anzahl der möglichen Ziehungen einer k-Permutation mit Wiederholung anzugeben, nutzen wir die Kurzform $P^*(n,k)$. Zum Beispiel ergeben sich für die Menge $M = \{0,1,2\}$ und $P^*(3,2) = 9$ die Folgen: $\langle 0,0\rangle$, $\langle 0,1\rangle$, $\langle 0,2\rangle$ $\langle 1,0\rangle$, $\langle 1,1\rangle$ $\langle 1,2\rangle$, $\langle 2,0\rangle$, $\langle 2,1\rangle$ und $\langle 2,2\rangle$ (vgl. [Witt 2013, 13]).

Zur Berechnung von $P^*(n,k)$ kann folgende Formel verwendet werden, vorausgesetzt, dass $n,k \geq 0$ (vgl. [Witt 2013, 13]).

$$P^*(n,k) = n^k$$

So ergibt sich beispielsweise für $P^*(3,2)$:

$$P^*(3,2) = 3^2 = 3 * 3 = 9$$

Beispiele für die Anwendungen für Permutationen finden sich in [Witt 2013]. Das Werk bietet auch eine kurze Zusammenfassung zum Thema Permutation [Witt 2013, 14].

2.4 Kombination ohne Wiederholung

In den bisher betrachteten kombinatorischen Strukturen ist die Anordnung relevant. Zwei Permutationen mit denselben Elementen, aber einer unterschiedlichen Anordnung sind verschieden. Ist dies nicht der Fall, wird von Kombinationen gesprochen. Beispielsweise sind die Kombinationen $\{1,2,3\}$ und $\{2,1,3\}$ nicht zu unterscheiden (vgl. [Witt 2013, 15]).

Definition k-Kombination ohne Wiederholung: Sei $M = [1,n]$ und $k \in \mathbb{N}_0$ mit $0 \leq k \leq n$. Dann kann aus der Menge M eine Teilmenge $T\{x_1, x_2, \dots, x_k\}$ mit $x_i \in M$ gewählt werden, wobei $x_i \neq x_j$ für $i \neq j, 1 \leq i,j \leq k$. Diese Teilmenge T entspricht einer k-Kombination ohne Wiederholung.

Eine k-Kombination ohne Wiederholung entspricht damit einer ungeordneten Ziehung Z von k Elementen aus der Menge M, ohne dass Elemente wiederholt werden dürfen.

Um die Anzahl der möglichen Ziehungen einer k-Kombination ohne Wiederholung anzugeben, nutzen wir die Kurzform $K(n,k)$. Zum Beispiel ergeben sich für die Menge $M = \{0,1,2\}$ und $P(3,2) = 3$ die Teilmengen: $\{0,1\}$, $\{0,2\}$ und $\{1,2\}$ (vgl. [Witt 2013, 15]).

Zur Berechnung von $K(n,k)$ kann folgende Formel verwendet werden, vorausgesetzt $0 \leq k \leq n$ gilt (vgl. [Witt 2013, 15]).

$$K(n,k) = \frac{n!}{k!\,(n-k)!} = \frac{n^{\underline{k}}}{k!}$$

So ergibt sich beispielsweise für $K(3,2)$:

$$K(3,2) = \frac{3!}{2!\,(3-2)!} = \frac{6}{2} = 3$$

Eine weitere gängige Notation verwendet den Binomialkoeffizienten.

$$\binom{n}{k} = \frac{n!}{k!\,(n-k)!} = \frac{n^{\underline{k}}}{k!}$$

Der Binomialkoeffizient wird als „n über k“ gesprochen und ist äquivalent zu den anderen Formeln (vgl. [Witt 2013, 16]). Eine weitere Darstellungsmöglichkeit ist die Produktnotation des Binomialkoeffizienten. Diese bietet eine algorithmisch effiziente Variante der Berechnung (vgl. [Graham et al. 1994, 153]).

$$\binom{n}{k} = \frac{n(n-1)\dots(n-k+1)}{k(k-1)\dots 1} = \prod_{i=1}^{k} \frac{n-i+1}{i} \tag{2.2}$$

2.5 Kombination mit Wiederholung

Wenn die Wiederholung von Elementen bei einer *k*-Kombination zulässig ist, wird dies wie folgt definiert.

Definition k-Kombination mit Wiederholung: Sei *M* eine Menge mit *n*-Elementen mit $k \in \mathbb{N}_0$. Dann kann aus der Menge *M* eine Teilmenge $T\,\{x_1, x_2, \dots, x_k\}$ mit $x_i \in M, 1 \leq i \leq k$ gewählt werden. Diese Teilmenge *T* definieren wir als *k*-Kombination mit Wiederholung.

Eine *k*-Kombination mit Wiederholung entspricht damit einer ungeordneten Ziehung von *k* Elementen aus der Menge *M*, wobei Elemente mehrfach vorkommen können.

Um die Anzahl der möglichen Ziehungen einer *k*-Kombination mit Wiederholung anzugeben, nutzen wir die Kurzform $K^*(n,k)$. Es ist möglich, eine *k*-Kombination mit Wiederholung als *k*-Kombination ohne Wiederholung zu definieren. Bei der Auswahl einer

solchen Kombination können $k-1$ Elemente erneut ausgewählt werden. Angenommen, diese Elemente wären seit Anfang an verfügbar gewesen, hätten die Menge $n+k-1$ Elemente. Wird diese in die Formel für eine *k*-Kombination ohne Wiederholung eingesetzt, ergibt sich die Anzahl der Möglichkeiten für eine *k*-Kombination mit Wiederholung (vgl. [Witt 2013, 16]).

Zur Berechnung von $K^*(n,k)$ kann die Formel für *k*-Kombination ohne Wiederholung verwendet werden, vorausgesetzt $n,k \geq 0$ (vgl. [Witt 2013, 13]).

$$K^*(n,k) = K(n+k-1,k) = \binom{n+k-1}{k} = \frac{(n+k-1)!}{k!\,(n-1)!} \tag{2.3}$$

Für die Menge $M = \{0,1,2\}$ und $K^*(3,2) = 6$ ergeben sich die Teilmengen: $\{0,0\}$, $\{0,1\}$, $\{0,2\}$, $\{1,1\}$, $\{1,2\}$ und $\{2,2\}$. Dabei ergibt sich für $K^*(3,2)$:

$$K^*(3,2) = \binom{3+2-1}{2} = \frac{(3+2-1)!}{2!\,(3-1)!} = 6$$

In [Witt 2013, 17] und [Berger 2023] wird die Formel zur Berechnung der Anzahl von *k*-Kombinationen mit Wiederholung zusätzlich unter Verwendung des steigenden Faktorielle definiert. Das steigende Faktorielle ist wie folgt definiert:

$$n^{\overline{k}} = (n+0)(n+1)(n+2)\ldots\big(n+(k-1)\big) = n(n+1)(n+2)\ldots(n+k-1)$$

Das steigende Faktorielle von *n* wird als „*n* hoch *k* steigend“ gesprochen (vgl. [Witt 2013, 5]). Die Formel wird definiert durch:

$$K^*(n,k) = \frac{n^{\overline{k}}}{k!} = \binom{n+k-1}{k} = \frac{(n+k-1)!}{k!\,(n-1)!}$$

Beispiele für die Anwendungen für Kombinationen finden sich in [Witt 2013]. Das Werk bietet eine kurze Zusammenfassung zum Thema Kombination [Witt 2013, 18].

3 Urnenmodell und Anwendungsfälle

In der Motivation wurden bereits Anwendungsfälle der Kombinatorik erwähnt. Sie alle folgen dem Muster: „Wie viele Möglichkeiten gibt es, k Objekte aus insgesamt n Objekten auszuwählen?“. Das vorangegangene Kapitel verdeutlicht, dass diese Frage nur anwendungsfallspezifisch beantwortet werden kann. Möglicherweise muss die Reihenfolge berücksichtigt werden oder Elemente können bei einer Ziehung mehrfach vorkommen. Oft ist das Objekt selbst irrelevant, entscheidend sind allein die Parameter. Dementsprechend lassen sich die Parameter aus der Frage abstrahieren und anhand eines simplen und universellen Modells beschreiben. Im folgenden Kapitel wird das Urnenmodell vorgestellt.

3.1 Grundbegriffe im Urnenmodell

In einem Modell wird ein Abschnitt der Realität abstrahiert, um sich auf die wesentlichen Merkmale zu konzentrieren. Das Ziel besteht darin, ein komplexes Problem auf einfach und verständlich darzustellen. Das Urnenmodell löst diese Aufgabe für einige Strukturen der Kombinatorik. Betrachtet wird folgendes Beispiel: „Wie viele Passwörter mit der Länge drei lassen sich aus dem Alphabet $A = \{a, b, c, d\}$ erzeugen?“. Der erste Schritt zur Erstellung des Modells besteht darin, die wichtigen Merkmale zu abstrahieren. Die ersten beiden sind leicht abzulesen. Auf der einen Seite die Anzahl der Objekte, die mit „n“ gekennzeichnet werden, auf der anderen Seite die Anzahl der Züge, d.h. wie oft eines der Objekte ausgewählt wird. Die Anzahl der Züge wird als k bezeichnet. Aus dem Problem geht hervor, dass $n = |A| = 4$ und $k = 3$. Die anderen beiden Merkmale, ob Wiederholungen erlaubt sind und ob die Anordnung der Elemente eine Rolle spielt, müssen aus dem Kontext erschlossen werden. Beides trifft auf ein Passwort zu, da einerseits das Passwort $\langle a, b, c\rangle$ ungleich dem Passwort $\langle c, b, a\rangle$ ist und andererseits das Passwort $\langle a, a, a\rangle$ gültig ist. Im zweiten Schritt werden die abstrahierten Merkmale auf das Urnenmodell übertragen. In dem Modell werden Kugeln aus einer Urne gezogen und in einer Schale gelegt. Das genannte Problem lässt sich leicht übertragen. Das Alphabet A entspricht der Urne, da es der Behälter ist, aus dem wir die Elemente entnehmen. Die Elemente $\{a, b, c, d\}$ entsprechen den Kugeln in der Urne. Als Nächstes werden k Kugeln aus der Urne gezogen und in eine Schale gelegt. Wenn die Reihenfolge eine Rolle spielt, hat die Schale $k = 3$ Plätze, von denen jeder eine Kugel fasst. Sollte die Reihenfolge unwesentlich sein, so hat die Schale nur einen Platz, in den alle Kugeln gelegt werden. Wenn Wiederholungen erlaubt sind, muss das Ergebnis der Kugel notiert werden, damit sie zurückgelegt werden kann. Unter der Annahme, dass Wiederholung nicht erlaubt sind, so kann die Kugel liegen bleiben. Basierend auf dem Wissen aus Kapitel 2.3 lässt sich darauf schließen, dass es sich um eine Permutation mit Wiederholung und

$P(4,3)$ handelt. Daraus ergibt sich, dass es $P(4,3) = 4^3 = 64$ verschiedene Ziehungen oder, im Kontext des Problems, 64 mögliche Passwörter gibt. Das Modell kann für eine Vielzahl von Problemen verwendet werden. Es unterstützt die vier in Kapitel 2 vorgestellten kombinatorischen Strukturen. In den folgenden Kapiteln wird für jede der vier Möglichkeiten ein Anwendungsfall aus der Informatik behandelt.

3.2 Auswahl mit Beachtung der Reihenfolge und ohne Wiederholungsmöglichkeit

Ein Problem, bei dem die Reihenfolge eine Rolle spielt und keine Wiederholungen möglich sind, könnte lauten: „Wie viele Möglichkeiten gibt es, eine Songplaylist $P = [1,5]$ zu durchmischen?“. Das Extrahieren der Merkmale gibt $n = |P| = 5$, und da alle Songs in der Playlist gemischt werden, ist auch $k = 5$. Die Reihenfolge, in welcher die Songs abgespielt werden, muss berücksichtigt werden. Songs sollten nicht zweimal abgespielt werden, daher gibt es keine Wiederholungen. Wenn die Merkmale auf das Modell übertragen werden, entsteht eine Urne mit 10 Bällen und einer Schale mit 10 Plätzen. Das bedeutet, dass $k = 5$ Kugeln aus einer Urne $U = [1,5]$ geordnet gezogen werden können, ohne sie zurückzulegen. Basierend auf Kapitel 2.2 kann darauf geschlossen werden, dass es sich um eine k-Permutation ohne Wiederholung handelt.

Zur Berechnung wird die entsprechende Formel für $P(5,5)$ verwendet:

$$P(5,5) = \frac{n!}{(n-k)!} = \frac{5!}{(5-5)!} = 120$$

Das bedeutet, dass es 120 verschiedene Möglichkeiten gibt, die Playlist anzuordnen.

3.3 Auswahl mit Beachtung der Reihenfolge und Wiederholungsmöglichkeit

Ein Problem, bei dem die Reihenfolge eine Rolle spielt und Wiederholungen möglich sind, wurde bereits durch das Passwort Beispiel gegeben. Es gibt jedoch noch eine weitere Besonderheit, die die Möglichkeit von Wiederholungen bietet. Betrachtet wird folgendes Problem: „Wie viele Passwörter mit der Länge vier lassen sich aus dem Alphabet $A = \{a, b\}$ erzeugen?“. Im Gegensatz zum Problem aus Kapitel 3.1 ist $k = 4 > 2 = n$, was bedeutet es werden öfter Kugeln aus der Urne gezogen, als sich Kugeln in der Urne befinden. Dies wird dadurch ermöglicht, dass die Kugeln wieder in die Urne zurückgelegt werden. Für kombinatorische Strukturen gilt daher Folgendes:

Mit Wiederholung: Ist Wiederholung erlaubt, dann kann $n \geq k$ oder $k \geq n$, unter der Bedingung $n, k \in \mathbb{N}$, sein.

Ohne Wiederholung: Ist Wiederholung nicht erlaubt, dann muss $n \geq k$, unter der Bedingung $n, k \in \mathbb{N}$, sein.

Die Anzahl der Passwörter lässt wie folgt berechnen:

$$P^*(2,4) = n^k = 2^4 = 16$$

Es gibt also 16 Möglichkeiten, ein vierstelliges Passwort mit den Buchstaben a und b zu bilden.

3.4 Auswahl ohne Beachtung der Reihenfolge und ohne Wiederholungsmöglichkeit

Beim maschinellen Lernen wird häufig eine Feature-Selection durchgeführt, um Overfitting des Modells zu verhindern. Eine beispielhafte Problemstellung könnte lauten: „Wie viele unterschiedliche Feature-Subsets gibt es, wenn nur 5 von 100 Features gewählt werden?“. Durch Extrahieren der Merkmale ergibt sich $n = 100$ und $k = 5$, da 5 von 100 Features selektiert werden. Da die Reihenfolge der gewählten Features keine Rolle spielt und Features nicht zweimal gewählt werden können, gibt es keine Wiederholungen und die Beachtung der Anordnung ist ebenfalls irrelevant. In der Terminologie des Urnenmodells bedeutet dies, dass aus einer Urne $U = [1, 100]$ ungeordnet fünf Kugeln gezogen werden und ohne diese zurückzulegen. Dies entspricht einer *k*-Kombination ohne Wiederholung.

Zur Berechnung wird die entsprechende Formel für $K(100, 5)$ verwendet:

$$K(100,5) = \frac{100!}{5!\,(100-5)!} = 75.287.520$$

Es gibt also $75.287.520$ verschiedene Feature-Subsets zum Trainieren des Modells.

3.5 Auswahl ohne Beachtung der Reihenfolge und Wiederholungsmöglichkeit

Der letzte Anwendungsfall des Urnenmodells beschreibt ein Problem, bei dem die Reihenfolge keine Rolle spielt und Wiederholungsmöglichkeit besteht. Ein solches Problem könnte lauten: „Wie viele unterschiedliche Spielwelten können generiert werden, wenn eine Welt aus 16 Gebieten besteht und diese aus der Menge $B =$

$\{Wüste, Gras, Meer, Jungel, Berge, Schnee\}$ gewählt werden?“ Bei der Entwicklung von Spielen wird häufig auf prozedurale Spielweltgenerierung gesetzt. Eine prozedurale Spielwelt verwendet oft einen Startwert (engl. Seed), um automatisch eine Welt aus verschiedenen vorgefertigten Teilen, basierend auf diesem zu erstellen. Eine Ziehung aus einer Urne könnte somit als Startwert für eine Welt dienen. Durch Extrahieren der Merkmale ergibt sich $n = |B| = 6$ und $k = 16$, da 16 Gebiete aus den acht Gebieten gewählt werden. Da die Reihenfolge der Gebiete keine Rolle spielt und jedes Gebiet mehrfach ausgewählt werden kann, kommt es zu Wiederholungen und einer irrelevanten Reihenfolge. Für das Urnenmodell bedeutet dies, dass 16 Kugeln aus einer Urne $U = [1, 6]$ ungeordnet gezogen werden und diese zurückgelegt werden. Es handelt sich daher um k-Kombination mit Wiederholung.

Zur Berechnung wird die entsprechende Formel für $K^*(6, 16)$ verwendet:

$$K^*(6, 16) = \binom{n+k-1}{k} = \frac{(n+k-1)!}{k!\,(n-1)!} = \frac{(6+16-1)!}{16!\,(6-1)!} = 20.349$$

Es können 20.349 unterschiedliche Welten generiert werden.

Eine ähnliche Erklärung des Urnenmodells mit anderen Beispielen außerhalb der Informatik findet sich in [Berger 2023, 30–39]. Darüber hinaus werden auf den folgenden Seiten weitere Materialien wie z.B. das Trennstrichmodell, im Englischen besser bekannt als „Stars and bars“, vorgestellt.

3.6 Zusammenfassung des Urnenmodells

Das Urnenmodell bietet eine einfache und verständliche Methode, mit k-Permutationen und k-Kombinationen umzugehen. Eine zusammenfassende Übersicht findet sich in Abbildung 1.

		Beachtung der Reihenfolge		
		Geordnete Auswahl	Ungeordnete Auswahl	
Wiederholungsmöglichkeit	Mit Zurücklegen	n^k	$\binom{n+k-1}{k} = \frac{(n+k-1)!}{k!\,(n-1)!} = \frac{n^{\overline{k}}}{k!}$	Immer möglich: $n \geq k$ oder $k \geq n$
	Ohne Zurücklegen	$n^{\underline{k}} = \frac{n!}{(n-k)!}$	$\binom{n}{k} = \frac{n!}{k!\,(n-k)!} = \frac{n^{\underline{k}}}{k!}$	Nur möglich für $k \leq n$

Abbildung 1: Übersicht Urnenmodell (in Anlehnung an [Berger 2023, 47])

4 Kombinatorische Unranking-Algorithmen

Das vorletzte Grundlagenkapitel behandelt Ranking- und Unranking-Algorithmen. Anschließend wird die lexikografische Ordnung erläutert, um schlussendlich in Kapitel 6 eine Verbindung zwischen den kombinatorischen Strukturen und den Unranking-Algorithmen herzustellen. Aus der Verbindung soll die Grundidee für die Implementierung des Prototyps hervorgehen.

4.1 Ranking- und Unranking-Algorithmen

Ranking- und Unranking-Algorithmen sind gegensätzliche Operationen und können als Hashfunktion für kombinatorische Strukturen Verwendung finden.

Definition Ranking: Der Rang eines Objektes ist die Position des Objektes, welche das Objekt in einer festgelegten Rangfolge hat. Ein Ranking-Algorithmus erzeugt aus einem gegebenen Objekt den eindeutigen Rang, den es in der Rangfolge hat (vgl. [Frank Ruskey 2003, 5]).

Definition Unranking: Unranking ist die Umkehroperation zum Ranking. Unranking ist der Prozess, bei dem aus einem Rang, dass an dieser Stelle der Rangfolge befindliche Objekt erzeugt wird (vgl. [Frank Ruskey 2003, 5–6]).

Zum besseren Verständnis lassen sich Ranking und Unranking auch mathematisch beschreiben.

Angenommen, es gibt eine endliche Menge *M* mit $n = |M|$ an kombinatorischen Strukturen. Dann kann eine bijektive Ranking-Funktion *rank* definieren werden, welche die Elemente von *M* auf einen Rang *R* abbildet:

$$rank: M \rightarrow \{0, 1, \dots, n-1\}$$

Dabei wird eine Ordnung durch folgende Regel vorgegeben:

$$m_1 < m_2 \Leftrightarrow rank(m_1) < rank(m_2)$$

Umgekehrt ist es möglich eine bijektive Unranking-Funktion *unrank* zu definieren, welche einen Rang *R* auf die Elemente von *M* abbildet:

$$unrank: \{0, 1, \dots, n-1\} \rightarrow M$$

Die Funktionen *rank* und *unrank* sind inverse Operationen. Somit gilt für alle Elemente $m \in M$ und den Rängen $r \in \{0, 1, \dots, n-1\}$:

$$rank(m) = r \Leftrightarrow unrank(r) = m$$

(vgl. [Kreher/Stinson 1999, 31–32]).

4.2 Anwendungsfälle für Ranking- und Unranking-Algorithmen

Der offensichtlichste Anwendungsfall für dies Klasse Algorithmen ist die Reduktion von benötigtem Speicherplatz. Für kombinatorische Strukturen wird beispielsweise eine *k*-Permutation ohne Wiederholung mit den Parametern $n, k = 6$ betrachtet. Das bedeutet, dass es $P(6, 6) = 720$ Permutationen gibt. Bei einem naiven Ansatz gäbe es ein Array mit 720 Positionen, das an jeder Stelle wiederum ein Array der Länge sechs speichert. Im Gegensatz dazu steht einem Integer-Array der Länge 720, das einen Unranking-Algorithmus nutzt. Ein allgemeinerer Anwendungsfall wäre die Zufallsgenerierung von Objekten mit Hilfe eines Unranking-Algorithmus. Steht für betreffendes Objekt ein Unranking-Algorithmus zur Verfügung, kann dieser in Verbindung mit einem Pseudozufallszahlengenerator verwendet werden, um zufällige Objekte zu erzeugen. Dies kann beispielsweise für das Benchmarking von Algorithmen nützlich sein (vgl. [Bender/S. Gill Williamson 2006, 75]).

4.3 Lexikografische Ordnung

Es gibt verschiedene Arten, eine Liste zu ordnen. Eine häufig verwendete Ordnung ist die lexikografische Ordnung. Die zugrundeliegende Idee ist dieselbe wie die Anordnung der Worte in einem Lexikon. Das bedeutet, dass die Ordnung des verwendeten Alphabets zur Bestimmung der Rangfolge der einzelnen Buchstaben herangezogen wird. Im Folgenden wird $\prec$ zum Vergleich der Ordnung des Alphabets und $<$ zum Vergleich von Indizes genutzt. Die Ordnung der Elemente des Alphabets entspricht den zugrundeliegenden Positionen im Alphabet, d.h. $0 \prec 1 \prec 2 \prec \cdots$.

Definition lexikografische Ordnung: In lexikografischer Ordnung gilt,

$a_1 a_2 \dots a_n < b_1, b_2 \dots b_m$, wenn für ein k, $a_k \prec b_k$ und $a_i = b_i$ für $i \in \{1, 2, \dots, k-1\}$

oder

$n < m$ und $a_i = b_i$ für $i \in \{1, 2, \dots, n\}$.

Zum Beispiel hat die Menge $M = \{224, 3, 134, 13, 21, 12\}$ die lexikografische Ordnung 12, 13, 134, 21, 224, 3. Andere Ordnungsmethoden, wie die umgekehrte lexikografische oder die co-lexikografische Ordnung, werden hier aufgrund ihrer mangelnden Relevanz für diese Arbeit nicht behandelt (vgl. [Frank Ruskey 2003, 55]).

5 Die Standard Template Library

Das letzten Grundlagenkapitel stellt die C++ Standard Template Library (STL) vor. Diese wird in Kapitel 7.2 als Referenzpunkt für die Erweiterung des Prototyps um einen STL-kompatiblen Iterator verwendet. In diesem Kapitel werden die einzelnen Komponenten der STL kurz erläutert, gefolgt von der Präsentation des Iteratoren Systems.

5.1 Komponenten der STL

Die Standard Template Library ist Teil der C++-Standardbibliothek und besteht aus drei Hauptkomponenten, die Optionen der Verwaltung von Daten mit effizienten Algorithmen bieten. Die erste der drei Komponenten sind Container, die Objektsammlungen verwalten. Die zweite Komponente, Iteratoren, wird verwendet, um diese Objektsammlungen schrittweise zu durchlaufen. Die letzte Komponente die Algorithmen wie Such- oder Sortieralgorithmen dienen zur Verarbeitung der Objektsammlungen. Vereinfacht ausgedrückt verwaltet der Container die Daten, die mithilfe der Algorithmen bearbeitet

werden können. Der Iterator dient als Verbindung zwischen dem Container und den Algorithmen. (vgl. [Josuttis 2012, 165–166]).

5.2 STL-Iteratoren

Iteratoren sind Objekte, die eine Objektsammlung schrittweise durchlaufen können. Sie zeichnen sich durch eine zu Grunde liegende Schnittstelle aus, die auf einem Pointer basiert. Die meisten Algorithmen der STL wie Sortieralgorithmen, benötigen zusätzlich spezielle Eigenschaft, um ordnungsgemäß zu funktionieren. Aus diesem Grund führt die STL das Konzept der Iterator-Kategorien ein. Jede Kategorie definiert ihre eigenen Eigenschaften und Fähigkeiten, die der Iterator erfüllen muss. Diese Kategorien bauen aufeinander auf (vgl. [Josuttis 2012, 433–434]). Abbildung 2 zeigt die Hierarchie der Iterator Kategorien und Tabelle 1 zeigt die Fähigkeiten jede Kategorie.

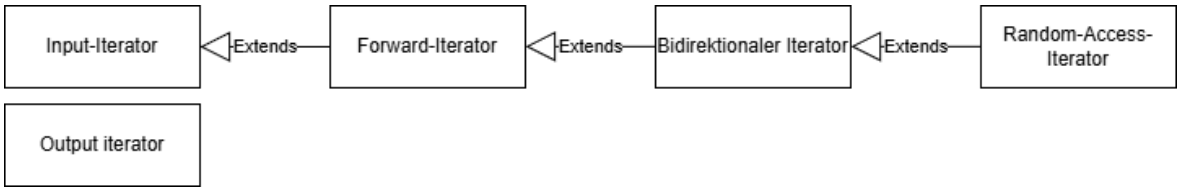

Abbildung 2: Hierarchie der Iterator Kategorien (in Anlehnung an [Josuttis 2012, 434])

Iterator Kategorie	**Aufgabe/Fähigkeit**
Output-Iterator	Vorwärts schreib
Input-Iterator	Einmaliges vorwärts lesen
Forward-Iterator	Vorwärts lesen
Bidirektionaler Iterator	Vorwärts und rückwärts lesen
Random-Access-Iterator	Lesen mit wahlfreiem Zugriff

Tabelle 1: Aufgaben/Fähigkeiten der Iterator-Kategorien (in Anlehnung an [Josuttis 2012, 434])

Eine detaillierte Erläuterung der einzelnen Iterator-Kategorien und ihre Implementierung würde den Rahmen dieser Arbeit überschreiten. Weitere Informationen sind unter anderem in [Josuttis 2012] zu finden. Josuttis Buch befasst sich mit dem C++-Standard. Die Quelle ist jedoch veraltet und umfasst lediglich den C++-Standard bis C++11. Seitdem haben sich die Anforderungen an die einzelnen Iterator-Kategorien geringfügig geändert und werden

seit C++20 über ein Konzeptsystem *(engl. concepts)* geregelt (vgl. [cppreference.com 2025a]). Die Konzepte selbst folgen derselben Vererbungshierarchie wie in Abbildung 2. Sie überprüfen, ob ein Iterator alle Eigenschaften und Fähigkeiten der angegebenen Iterator-Kategorie aufweist.

6 Unranking-Algorithmen für das Urnenmodell

Nach der Einführung in die Grundlagen konzentriert sich der restliche Teil der Bachelorarbeit auf eine prototypische Implementierung für das Urnenmodell. Ein naiver Ansatz wäre, die kombinatorischen Strukturen mittels iterativen Ansatzes zu generieren. Wie jedoch in der Problemstellung und Motivation erwähnt, ist dies nicht besonders effektiv. Dementsprechend werden zur Generierung der kombinatorischen Strukturen Unranking-Algorithmen verwendet. Da es in der Literatur viele verschiedene Ansätze für kombinatorische Unranking-Algorithmen gibt, werden einige Restriktionen auferlegt. Um die Vergleichbarkeit mit den kombinatorischen Algorithmen der C++-Standardbibliothek wie *std::next_permutation* (vgl. [cplusplus.com 2025]) zu gewährleisten, werden nur lexikografische Algorithmen betrachtet. Diese Restriktion schließt somit Algorithmen der umgekehrten lexikografischen oder der co-lexikografischen Ordnung aus. Auch Graycode-Algorithmen, die ihre eigene Ordnung festlegen, werden ausgeschlossen, auch wenn diese geringere Zeit- oder Speicherkomplexität aufweisen (vgl. [Frank Ruskey 2003, 115]). Eine weitere Restriktion ist die Beschränkung auf die vier Grundtypen des Urnenmodells. Als weiterführende Literatur wird [Curiel/Genitrini 2024] empfohlen. Dort werden für alle zwölf kombinatorischen Strukturen des Twelvefold Way Unranking-Algorithmen, eingeschlossen der vier Grundtypen, vorgestellt und auf weitere Literatur verwiesen. In diesem Kapitel werden vier Unranking-Algorithmen für k-Permutation und k-Kombination vorgestellt, jeweils mit und ohne Wiederholung.

6.1 Unranking für Permutation ohne Wiederholung

Es existieren zahlreiche Unranking-Algorithmen für Permutationen ohne Wiederholung, aber diese verwendet oft die klassische Form einer Permutation, die keine Auswahl einer Teilmenge der Länge k zulässt, sondern $n = k$ erfordert. Ein Beispiel hierfür ist [Myrvold/Ruskey 2001]. Ihre Arbeit präsentiert Ranking- und Unranking-Algorithmen für klassische Permutationen, die in linearer Zeit laufen. Im Folgenden wird ein Algorithmus für k-Permutation ohne Wiederholung aus einem Algorithmus für klassische Permutationen abgeleitet.

Der zugrundeliegende Ansatz basiert auf der Idee des Lehmer-Codes, der nach D. H. Lehmer benannt ist. Die mathematischen Grundlagen wurden jedoch bereits im 19. Jahrhundert in [Laisant 1888] untersucht (vgl. [Berger et al. 2019, 7]). Der Lehmer-Code wird in [Berger et al. 2019] vorgestellt. Er kann zum Ranking von Permutationen verwendet werden und basiert mathematisch auf den Konzepten der Inversion von Permutationen und dem fakultätsbasierten Zahlensystem. Um die Herleitung des Algorithmus für *k*-Permutationen zu verstehen, werden im Folgenden die beiden Konzepte und der Lehmer-Code vorgestellt.

Angenommen, es gibt eine Permutation ohne Wiederholung aus einer Menge mit *n*-Elementen. Eine beliebige Permutation ist dann z.B. $P\ \langle x_1, x_2, \dots, x_n \rangle$. Wenn auf ein Elementenpaar der Permutation $i < j$ und $x_i > x_j$ zutrifft, dann ist das Paar (x_i, x_j) eine Inversion der Permutation. Beispielsweise hat die Permutation $\langle 4,1,2,5,3 \rangle$ vier Inversionen, nämlich $(4,1), (4,2), (4,3)$ und $(5,3)$ (vgl. [Knuth 1997, 11]). Mit dem Wissen, was Inversionen einer Permutation sind, kann eine Inversionstabelle für die Permutation erstellt werden. Es gibt verschiedene Ansätze zur Erstellung von Inversionstabellen. Die bekanntesten sind die Ansätze nach Knuth (vgl. [Knuth 1997, 11–12]) und Lehmer. Es wird dem Ansatz von Lehmer gefolgt. Die Inversionstabelle des Lehmer-Codes $l := l_1 l_2 \dots l_n$ einer Permutation $P\ \langle x_1, x_2, \dots, x_n \rangle$ hat ebenfalls *n* Elemente. Die Elemente der Inversionstabelle sind wie folgt definiert:

$$l_i := \left| \{ j > i \mid x_j < x_i \} \right|$$

Einfach ausgedrückt ist l_i die Anzahl der Elemente von $P\ \langle x_1, x_2, \dots, x_n \rangle$, die rechts von x_i stehen und kleiner als dieses sind. Beispielsweise lautet die Inversionstabelle der Permutation $\langle 4,1,2,5,3 \rangle$ $l := 3,0,0,1,0$. Das erste Element der Inversionstabelle ist $l_1 = 3$, da rechts neben 4 in der Permutation die 1, 2 und 3 kleiner als 4 sind. Für die Elemente 1, 2 der Permutation gibt es keine Elemente rechts von ihnen, die kleiner sind, daher sind die nächsten beiden Elemente der Inversionstabelle $l_2, l_3 = 0$. Rechts von 5 steht 3, die kleiner ist als 5, daher ist l_4=1. Das letzte Element der Inversionstabelle ist wieder 0, da es kein anderes Element rechts von 3 gibt (vgl. [Höfler 2018, 38]). Die Inversionstabelle ist für ihrer Permutation eindeutig. Mit der Tabelle lässt sich die ursprüngliche Permutation rekonstruieren. Um diese zu erhalten, wird sukzessiv aus einer Liste der Elemente der Permutation $E = [1,2, \dots, n]$ immer, dass (l_i+1) -kleinste noch nicht verwendete Element entfernt. Für die Beispiel Permutation $\langle 4,1,2,5,3 \rangle$ und ihre Inversionstabelle $l := 3,0,0,1,0$ wäre die Liste $E = [1,2,3,4,5]$. Als nächstes kann die ursprüngliche Permutation mittels der

Inversionstabelle rekonstruiert werden. Zunächst wird das (3+1) -kleinste noch nicht verwendete Element aus der Liste entfernt. Die neue Liste ist $E = [1,2,3,5]$ und es wird mit dem nächsten Element fortgefahren. Das zweite Element der Permutation ist durch das (0+1) -kleinste noch nicht verwendete Element aus der Liste E, also die 1 gegeben. Diese wird anschließend aus der Liste entfernt. Für das dritte Element der Permutation wird das (0+1) -kleinste noch nicht verwendete Element ausgewählt, d.h. 2, und ebenfalls aus der Liste entfernt. Element vier der ursprünglichen Permutation ergibt sich aus dem (1+1) -kleinste noch nicht verwendete Element aus der Liste $E = [3,5]$, d.h. 5. Schließlich wird die 3 als letztes Element aus der Liste gewählt. Mit diesem Verfahren lässt sich die Beispiel Permutation $\langle 4,1,2,5,3 \rangle$ aus der Inversionstabelle rekonstruieren (vgl. [Höfler 2018, 38–39]).

Das fakultätsbasierte Zahlensystem findet häufig Anwendung in Verbindung mit Permutationen. Eine Fakultätszahl hat n Elemente und somit die Form $F = x_{n-1}x_{n-2} \dots x_i \dots x_1 x_0$ mit $0 \leq x_i \leq i, 0 \leq i \leq n-1$. Um diese Zahl im Dezimalsystem darzustellen, kann folgende Formel verwendet werden:

$$Z = x_{n-1}(n-1)! + x_{n-2}(n-2)! + \cdots + x_1 1! + x_0 0! \tag{6.1}$$

wobei Z die Dezimalzahl ist (vgl. [Borysenko et al. 2024, 3]). Mit dieser Formel lässt sich beispielsweise die Fakultätszahl 3,0,0,1,0 durch die Rechnung $3 * 4! + 0 * 3! + 0 * 2! + 1 * 1! * 1 * 0!$ zur Dezimalzahl 73 umwandeln. Der Ursprung des Zahlensystems ist nicht ganz klar, aber es wird bereits 1888 in [Laisant 1888, 176] erwähnt.

Wie zu Beginn des Kapitels erwähnt, wird der Lehmer-Code in [Berger et al. 2019] vorgestellt. Mit Formel 7 findet sich konzeptionell die Definition von Inversionen von Permutationen wieder (vgl. [Berger et al. 2019, 6]). Der Lehmer-Code generiert dann anhand der Inversionstabelle und der Idee aus Formel (6.1) einen Rang für eine gegebene Permutation.

Die Formel lautet:

$$rank = \sum_{i=1}^{n} l_i (n-i)!, \text{ sodass } rank \in [0, n!-1] \tag{6.2}$$

Sie kombiniert die Elemente der Inversionstabelle zu einer Zahl, die als Rang dient (vgl. [Berger et al. 2019, 7]). Für die Permutation $\langle 4,1,2,5,3 \rangle$ und ihren Lehmer-Code 3,0,0,1,0 würde sich als Ergebnis der Rang 73 ergeben. Die Formel kann direkt als Ranking-

Algorithmus für klassische Permutation genutzt werden. Im nächsten Schritt kann aus diesem die Umkehroperation, ein Unranking-Algorithmus, abgeleitet werden. Für die Konstruktion ist es notwendig, eine Abfolge von Schritten festzulegen, mit denen die Inversionstabelle aus dem Rang generiert werden kann. Mit Hilfe der Inversionstabelle kann dann die entsprechende Permutation nach dem obigen Beispiel rekonstruiert werden. Die ausgeschriebene Ranking-Formel des Lehmer-Codes dient dabei als Ansatz:

$$rank = l_1(n-1)! + l_2(n-2)! + \cdots + l_{n-1}1! + l_n 0!$$

Die folgende Formel wird verwendet, um die Elemente der Inversionstabelle sukzessiv in einer Schleife zu berechnen:

$$l_i = \lfloor rank/(n-i)! \rfloor$$

In jeder Schleifeniteration muss der *rank* neu definiert werden, damit er für die nächste Iteration verfügbar ist.

$$rank := rank\ mod\ (n-i)!$$

Auf diese Weise kann die Inversionstabelle des Lehmer-Codes konstruiert werden. Für die Beispiel Permutation $\langle 4,1,2,5,3 \rangle$ und ihren Rang 73 würde es folgendermaßen aussehen:

Iteration 1: $l_1 = \lfloor 73/(5-1)! \rfloor = 3 \mid rank := 73\ mod\ (5-1)! = 1$

Iteration 2: $l_2 = \lfloor 1/(5-2)! \rfloor = 0 \mid rank := 1\ mod\ (5-2)! = 1$

Iteration 3: $l_3 = \lfloor 1/(5-3)! \rfloor = 0 \mid rank := 1\ mod\ (5-3)! = 1$

Iteration 4: $l_4 = \lfloor 1/(5-4)! \rfloor = 1 \mid rank := 1\ mod\ (5-4)! = 0$

Iteration 5: $l_5 = \lfloor 0/(5-5)! \rfloor = 0 \mid rank := 0\ mod\ (5-5)! = 0$

Nachdem alle Iterationen abgeschlossen sind, ergibt sich als Ergebnis der Lehmer-Code $l = 3,0,0,1,0$. Aus diesem kann die Permutation rekonstruiert werden. Die hier gezeigte Methode dient als Unranking-Algorithmus für klassische Permutationen.

```
1. function unrank_permutations_with(n, rank)
 2.     rest = rank
 3.     permutation = []
 4.     elements = [1…n]
 5.     for (i =1; i<=n; ++i)
 6.         denominator = factorial(n-i)
 7.         inversion_number = rest div denominator
 8.         permutation.append(elements.pop(inversion_number))
 9.         rest = rest mod denominator
10.     return permutation
11.
```

Algorithmus 1: Unranking Permutation

Ein ähnlicher Ansatz unter Verwendung einer Abwandlung von Formel (6.1) wird in [Genitrini/Pépin 2021] vorgestellt.

Für eine *k*-Permutation ohne Wiederholung muss der Algorithmus leicht angepasst werden. Die for-Schleife läuft nicht mehr *n*-mal, sondern nur noch *k*-mal. Auch der Divisor aus Zeile 6 ändert sich. Er wird zum fallenden Faktorielle und wird mit Formel (2.1) berechnet. Für den Unranking-Algorithmus für *k*-Permutation ohne Wiederholung ergibt sich folgender C++-Code:

```
 1. std::vector<int> unrank_permutations_without_repetition(int n, int k, int
rank)
 2. {
 3.     std::vector<int> elements {};
 4.     for (int i {1}; i <= m_n; ++i) {
 5.         elements.push_back(i);
 6.     }
 7.
 8.     std::vector<int> permutation {};
 9.     int rest {rank};
10.
11.     for (int i {1}; i <= m_k; ++i) {
12.         int denominator = falling_factorial(n-i, k-i);
13.         int inversion_number = rest / denominator;
14.         permutation.push_back(elements[inversion_number] - 1);
15.         elements.erase(elements.begin() + inversion_number);
16.
17.         rest = rest % denominator;
18.     }
19.
20.     return permutation;
21. }
22.
```

Algorithmus 2: Unranking k-Permutation ohne Wiederholung in C++

6.2 Unranking für Permutation mit Wiederholung

Ein Unranking-Algorithmus für Permutationen mit Wiederholung lässt sich aus dem Konzept der Basisexpansion ableiten. Die Basisexpansion ist ein Konzept der Integerdarstellung und wird verwendet, um Dezimalzahlen (Basis 10) in eine Zahl mit beliebiger Basis umzuwandeln.

Sei $n, b \in \mathbb{N}, n > 0$ und $b > 1$ dann kann n wie folgt dargestellt werden:

$$n = a_k b^k + a_{k-1} b^{k-1} + \cdots + a_1 b + a_0$$

wobei k und $a_0, a_1, .., a_k \in \mathbb{N}_0, a_k \neq 0$ und $a_0, a_1, .., a_k > b$ (vgl. [Rosen 2012, 245–246]).

Bei einer Basisexpansion wird die Zahl n sukzessive durch die Basis b geteilt. Dabei werden jeweils der Quotient q und der Rest der Division r ermittelt. Der Quotient wird erneut durch b geteilt, bis er den Wert Null erreicht. Die so erhaltenen Reste ergeben in umgekehrter Reihenfolge die Zahl in der gewünschten Basis.

Ein Algorithmus für die Basisexpansion findet sich in Algorithmus 3 . Die *mod*-Operation entspricht der Modulo Operation, die den Rest einer Ganzzahldivision ermittelt. Die *div*-Operation entspricht einer Ganzzahldivision, die die beiden Zahlen dividiert und das Ergebnis auf die nächstkleinere Zahl abrundet.

```
1. function base_expansion(n, b)
 2.      q = n
 3.      k = 0
 4.      a = []
 5.      while q ≠ 0
 6.              a.append(q mod b)
 7.              q = q div b
 8.              k = k + 1
 9.      return a
```

Algorithmus 3: Basis-Expansion (in Anlehnung an [Rosen 2012, 249])

Beispielsweise sind für $n = 12345$ und $b = 8$ die Reste 1, 7, 0, 0, 3, was in umgekehrter Reihenfolge die Oktalzahl 30071 ergibt (vgl. [Rosen 2012, 247–248]). Die Reste werden durch die folgende Berechnung ermittelt:

$$12345 = 8 * 1543 + 1$$

$$1543 = 8 * 192 + 7$$

$$192 = 8 * 24 + 0$$

$$24 = 8 * 3 + 0$$

$$3 = 8 * 0 + 3$$

Algorithmus 3 kann verwendet werden, um einen Unranking-Algorithmus für Permutationen mit Wiederholung abzuleiten. Ausgangspunkt ist die Definition einer Unranking-Funktion, aus Kapitel 4.1, die besagt, dass jede Permutation durch einen Rang abgebildet werden kann. Angenommen, es ist die Ziehung mit dem Rang r aus einer $P(n, k)$

gesucht. Wenn der Rang r als Nummer und die Anzahl unserer Elemente n als die Basis in Algorithmus 3 gegeben, kann die entsprechende Permutation mit Wiederholung konstruiert werden. Hier wird r in Basis n dargestellt. Die resultierenden Reste, die in der Liste gespeichert sind, ergeben die gewünschte Permutation mit Wiederholung in umgekehrter Reihenfolge. Diese muss mit führenden Nullen ergänzt werden, um die Länge k zu erreichen.

Um Algorithmus 3 zu optimieren, kann die while-Schleife durch eine rückwärtslaufende for-Schleife ersetzt werden, um die Ziehung direkt in der richtigen Reihenfolge anzuzeigen. Außerdem kann die Liste zunächst mit k Nullen gefüllt werden und immer am erforderlichen Index geändert werden. Für den Unranking-Algorithmus für Permutation mit Wiederholung ergibt sich somit folgender C++-Code:

```
 1. std::vector<int> unrank_permutations_with_repetition(int n, int k, int r)
 2. {
 3.     std::vector<int> permutation(m_k, 0);
 4.     for (int i {k - 1}; i >= 0; --i)
 5.     {
 6.         permutation[i] = r % n;
 7.         r /= n;
 8.     }
 9.     return permutation;
10. }
11.
```

Algorithmus 4: Unranking k-Permutation mit Wiederholung in C++ (in Anlehnung an [Rosen 2012, 249])

6.3 Unranking für Kombination ohne Wiederholung

In [Genitrini/Pépin 2021] präsentieren Genitrini und Pépin eine Reihe von Unranking-Algorithmen für verschiedene kombinatorische Strukturen. In Kapitel 2 stellen sie Unranking-Algorithmen für Kombinationen ohne Wiederholung aus der Literatur vor und erläutern eine neue Methode unter der Verwendung des fakultätsbasierten Zahlensystems. In ihrem Ansatz definieren sie zunächst eine Bijektion zwischen der Menge an k-Kombinationen ohne Wiederholung und der Menge an Permutationen für n Elemente. Anschließend wandeln sie den Rang r der k-Kombinationen ohne Wiederholung in den Rang r^p der entsprechenden Permutation um. Schließlich verwenden sie einen Unranking-Algorithmus für Permutationen, um die Permutation des Ranges r^p zu generieren und das Präfix der Länge k zu extrahieren. Dieses Präfix entspricht dann der k-Kombination ohne Wiederholung des Ranges r (vgl. [Genitrini/Pépin 2021, 6]). Dieser Ansatz führt zu folgendem Algorithmus 5.

```
1. function unrank_combination_without_repetition(n, k, r)
2.     rank_permutation = rankconversion(n,k,r)
3.     permutation = unrank_permutation(n, rank_permutation)
4.     return [permuation[0]...permutation[k-1]]
5.
```

Algorithmus 5: Unranking k-Kombinationen ohne Wiederholung (in Anlehnung an [Genitrini/Pépin 2021, 7])

Zeile 2 entspricht der Umwandlung von Rang r zu Rang r^p. Die Funktion *rankconversion* ist durch den Pseudocode aus Algorithmus 6 gegeben. Ein Beweis findet sich in [Genitrini/Pépin 2021, 6–7]. Die Funktion *unranking_permutation* ist durch ihren Algorithmus 1 gegeben (vgl. [Genitrini/Pépin 2021, 5]).

```
 1. function rankconversion(n, k, r)
 2.     F = [0] * n
 3.     i = 0
 4.     m = 0
 5.     while i < k
 6.         b = binomial(n-1-m-i, k-1-i)
 7.         if b > u
 8.             F[n-1-i] = m
 9.             i += 1
10.         else
11.             u -= b
12.             m += 1
13.     return factoradic_to_number(F)
14.
```

Algorithmus 6: Rangkonvertierung (in Anlehnung an [Genitrini/Pépin 2021, 7])

Die Funktion *factoradic_to_number* wandelt die Fakultätszahl F in die entsprechende dezimale Zahl um und entspricht dem Verfahren in Formel (6.1). Ein konkreter Beweis des Algorithmus wird in [Genitrini/Pépin 2021, 6–7] vorgestellt. Kapitel 4.2 ihrer Arbeit beschreibt zusätzliche Strategien zur Optimierung von Algorithmus 5. Die optimierte Version von Algorithmus 5 wird als C++-Algorithmus in Algorithmus 7 präsentiert.

```
1. std::vector<int> unrank_combination_without_repetition(int n, int k, int r) {
 2.     std::vector<int> combination(k, 0);
 3.
 4.     int binomial = binomial_coefficient(n-1, k-1) * n;
 5.     int m {0};
 6.     int index {0};
 7.
 8.     while (index < k - 1) {
 9.         binomial = binomial / (n - m - index);
10.
11.         if (binomial > r) {
12.             combination[index] = m + index;
13.             binomial = binomial * (k - index - 1);
14.             ++index;
15.         } else {
16.             r -= binomial;
17.             binomial = binomial * (n - m - k);
18.             ++m;
19.         }
20.     }
21.
22.     if (k > 0) {
23.         combination[k - 1] = n + r - binomial;
24.     }
25.
26.     return combination;
27. }
28.
```

Algorithmus 7: Unranking k-Kombinationen ohne Wiederholung in C++ (in Anlehnung an [Genitrini/Pépin 2021, 20])

Eine der zeitaufwendigsten Operationen ist die Berechnung des Binomialkoeffizienten. In Algorithmus 6 wird dieser für jede Iteration der while-Schleife neu berechnet, obwohl sich die Parameter gegenüber der vorherigen Iteration nur um ± 1 unterscheiden. Genitrini und Pépin schlagen eine Variante vor, bei der der Binomialkoeffizient auf das vorherige Ergebnis Bezug nimmt. In ihrer verbesserten Variante wird die komplexe Berechnung auf eine konstante Anzahl von Multiplikation oder Division durch eine kleine ganze Zahl reduziert. Je nachdem, welcher Zweig der if-Verzweigung ausgewählt wird, wird eine zusätzliche Zeile hinzugefügt, um den Binomialkoeffizienten b neu zu berechnen. Die Unterschiede werden deutlich, in einem direkten Vergleich von Algorithmus 6 und Algorithmus 7. In Algorithmus 7 wird der Binomialkoeffizient nur einmal vor dem Start der while-Schleife in Zeile 4 berechnet. Er wird dann innerhalb der Schleife durch die Berechnungen in den Zeilen 9, 13 und 17 angepasst.

Als weitere Verbesserung verwenden sie nicht die Reihenfolge aus Algorithmus 5, sondern kombinieren den Prozess so, dass die richtigen Werte direkt aus der Menge $\{0,1,2,..,n-1\}$ ausgewählt werden. Außerdem entfernen sie die Menge der gespeicherten Elemente aus ihrer *unranking_permutation* Funktion (vgl. [Genitrini/Pépin 2021, 5]), da dies ein Sonderfall ist, in dem die entfernten Elemente in aufsteigender Reihenfolge ausgewählt werden. Die

Elemente werden dann durch m+i in Zeile 12 berechnet, wobei i die Zahl der bereits ausgewählten Elemente angibt. Schließlich kann das letzte Element ohne weitere Schleifeniteration berechnet werden, da die Berechnung des letzten Binomialkoeffizienten nicht mehr erforderlich ist (vgl. [Genitrini/Pépin 2021, 19]).

6.4 Unranking für Kombination mit Wiederholung

Schließlich wird ein Unranking-Algorithmus für Kombination mit Wiederholung vorgestellt, der auf den Überlegungen in Kapitel 2.5 und Algorithmus 7 basiert. In Kapitel 2.5 wird erläutert, dass es möglich ist, die Anzahl der möglichen Ziehungen einer *k*-Kombination mit Wiederholung als *k*-Kombination ohne Wiederholung zu bestimmen, indem die Menge der auszuwählenden Elemente auf $n + k - 1$ Elemente erweitert wird. Als Beispiel wird in Formel (2.3) $n = 2$ und $k = 3$ eingesetzt. Daraus wird deutlich, dass $K(4,3) = K^*(2,3) = 4$ gleich groß sind. Die möglichen Ziehungen sind in Tabelle 2 dargestellt.

$K(n + k - 1, k) = K(4,3)$	$K^*(n, k) = K^*(2, 3)$
$\{0,1,2\}$	$\{0,0,0\}$
$\{0,1,3\}$	$\{0,0,1\}$
$\{0,2,3\}$	$\{0,1,1\}$
$\{1,2,3\}$	$\{1,1,1\}$

Tabelle 2: Vergleich der Ziehungen für K(4,3) und K(2,3)*

Tabelle 2 zeigt deutlich, dass sich die Ziehungen nur hinsichtlich ihres Positionsindex unterscheiden. Jedes Element der Ziehung $\{1,2,3\}$ wird um seinen nullbasierten Positionsindex reduziert, was zur Ziehung der entsprechenden *k*-Kombination mit Wiederholung führt, d.h. $\{1,1,1\}$. Dieses Phänomen ist kein Zufall und gilt für jede Ziehung. Natürlich kann diese Transformation auch umgekehrt angewandt werden, um von einer *k*-Kombination mit Wiederholung zu einer *k*-Kombination ohne Wiederholung zu gelangen. Diese Methode ist daher bijektiv. Der Beweis dafür findet sich in [Witt 2013, 17]

```
1. function unrank_combination_with_repetition(n, k, r)
2.     N = n + k - 1
3.     combination = unrank_combination_without_repetition(N, k, r)
4.     for (i=0; i < k; ++i)
5.         combination[i] -= i
6.     return combination
7.
```

Algorithmus 8: Unranking k-Kombinationen mit Wiederholung

Diese Generierungsmethode ist auch deshalb vorteilhaft, weil sie mit jedem Unranking-Algorithmus für k-Kombinationen ohne Wiederholungen verwendet werden kann. Algorithmus 8 zeigt ein Beispiel der Implementierung der Methode. Da diese Variante zusätzlich zum eigentlichen Unranking-Algorithmus eine k-lange for-Schleife durchläuft, ist Algorithmus 8 nicht besonders effizient. Aus diesem Grund stellt Algorithmus 9 eine modifizierte Version von Algorithmus 7 für k-Kombinationen mit Wiederholung dar.

```
1. std::vector<int> unrank_combination_with_repetition(int n, int k, int r) {
 2.     std::vector<int> combination(k, 0);
 3.
 4.     int binomial = binomial_coefficient(n + k - 2, k - 1) * (n + k - 1);
 5.     int m = 0;
 6.     int index = 0;
 7.
 8.     while (index < k - 1) {
 9.         binomial = binomial / (n + k - 1 - m - index);
10.
11.         if (binomial > r) {
12.             combination[index] = m;
13.             binomial = binomial * (k - index - 1);
14.             ++index;
15.         } else {
16.             r -= binomial;
17.             binomial = binomial * (n - 1 - m);
18.             ++m;
19.         }
20.     }
21.
22.     if (n + k - 1 > 0) {
23.         combination[k - 1] = n + r - binomial;
24.     }
25.
26.     return combination;
27. }
28.
```

Algorithmus 9: Unranking k-Kombinationen mit Wiederholung in C++

Algorithmus 9 entspricht im Allgemeinen Algorithmus 7. In den Zeilen 4, 9, 17 und 22 wird n durch $n + k - 1$ ersetzt. Die Zuweisung in Zeile 12 ändert sich von $m + index$ zu nur m. Zeile 23 ändert sich nicht, da die Zuweisung des letzten Elements $(n + k - 1 + r - binomial) - (k - 1)$ zu $n + r - binomial$ ausgewertet wird.

7 Konzeption und Implementierung des Prototyps

Unter der Verwendung der in Kapitel 6 vorgestellten Algorithmen wird als Nächstes ein Prototyp des Urnenmodells entwickelt. Das Kapitel ist in zwei Teile gegliedert. Zunächst werden die Grundstruktur des Prototyps und seine Bestandteile erläutert. Im zweiten Teil wird der Prototyp dann um einen STL-konformen C++-Iterator erweitert.

7.1 Prototyp des Urnenmodells in C++

Das Ziel des Prototyps ist es, das in Kapitel 3 beschriebene Konzept des Urnenmodells umzusetzen. Insbesondere soll der Prototyp die grundlegende Struktur und Funktionalität einer Urne widerspiegeln. Als funktionale Anforderung muss der Prototyp die Verwaltung der vier Urnentypen und des Ziehungsprozesses ermöglichen. Dazu gehört insbesondere die Unterscheidung der spezifischen Unranking-Algorithmen für jeden Urnentyp. Zusätzlich zu den funktionalen Anforderungen soll der Prototyp auch nicht-funktionale Anforderungen erfüllen. Erstens soll der Prototyp leicht erweiterbar sein. Zweitens soll jeder Urnentyp über eine identische Schnittstelle verfügen, um die Verwendung zu vereinfachen. Schließlich sollen die Urnentypen einheitliche und intuitive benannt werden.

Um die Anforderungen zu erfüllen, wird der Prototyp des Urnenmodells als objektorientiertes System konzipiert. Eine abstrakte Basisklasse eignet sich für die beiden nicht-funktionalen Anforderungen der Erweiterbarkeit und identischer Schnittstellen. Diese kann eine gemeinsame Schnittstelle festlegen und die Erweiterung zukünftiger Änderungen am Code erleichtern. Um die Namen der vier Urnentypen möglichst so kurz und intuitiv wie möglich zu gestalten, können die Merkmale der Anordnung und Wiederholung jedes Typs verwendet werden. So enthält im Folgenden eine Urne, bei der die Anordnung eine Rolle spielt, das Suffix O *(für engl. „Order“)*. Eine Urne, bei der Wiederholungen zulässig sind, erhält das Suffix R *(für engl. „Repetition“)*. Eine Urne für *k*-Permutation mit Wiederholung erhält somit den Klassennamen *UrnOR*, eine Urne für *k*-Permutation ohne Wiederholung den Namen *UrnO*, eine Urne für *k*-Kombination mit Wiederholung den Klassennamen *UrnR* und eine Urne für *k*-Kombination ohne Wiederholung nur den Namen *Urn*. Eine Übersicht über den Prototyp bietet das UML-Klassendiagramm in Abbildung 3.

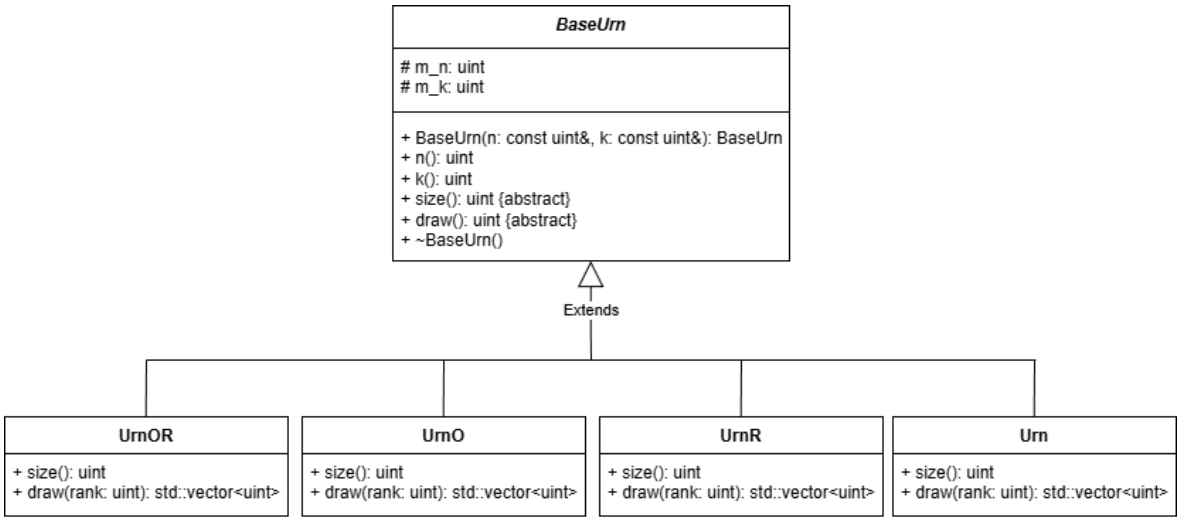

Abbildung 3: UML-Klassendiagramm des Urnenmodell-Prototyps

Abbildung 3 zeigt deutlich die Vererbungshierarchie. Obwohl die vier Urnentypen inhaltlich miteinander verwandt sind, ist eine Vererbung zwischen den Subklassen nicht angemessen. Eine solche Vererbungshierarchie würde implizieren, dass die einzelnen Urnentypen ein gemeinsames Merkmal haben, wie beispielsweise den Unranking-Algorithmus. Dies ist zwar für *k*-Kombinationen der Fall, da Algorithmus 9 von Algorithmus 7 abgeleitet ist, aber bei *k*-Permutationen besteht dieser Zusammenhang nicht. Die einzige Gemeinsamkeit zwischen den vier Urnentypen besteht in ihrer Konstruktor Schnittstelle und den Parametern *n* und *k*, die wiederum alle Urnentypen gemeinsam haben. Dementsprechend ist eine abstrakte Klasse für diesen Zweck besser geeignet.

Die abstrakte Klasse *BaseUrn* definiert eine einfache Schnittstelle für die vererbenden Subklassen. Die Urnen werden mit *n*, d.h. der Anzahl der Objekte, aus denen eine Ziehung erfolgt und *k,* d.h. der Anzahl der Objekte, die eine Ziehung hat, initialisiert und als nicht-negativer Integer gespeichert. Diese beiden Werte können mit der jeweiligen Zugriffsfunktion *(engl. Getter)* abgerufen werden. Darüber hinaus stellt die Basisklasse zwei weitere abstrakte Methoden zur Verfügung, die jeweils in den Subklassen implementiert sind. Die Funktion *size()* berechnet die Anzahl der möglichen *k*-Permutationen bzw. *k*-Kombinationen und gibt diesen Wert zurück. Zur Berechnung der Rückgabewerte werden drei Funktionen verwendet. Für UrnO wird eine direkte Implementierung des fallende Faktorielles basierend auf der Formel (2.1) implementiert. Für die *size()*-Funktion von UrnOR wird die *std::pow()*-Funktion der C++-Standardbibliothek aus dem Header cmath verwendet (vgl. [cppreference.com 2025c]). Für Urn und UrnR wird die Produktformel des Binomialkoeffizienten implementiert, die Formel (2.2) entspricht. Der Quellcode ist in Anhang A zu finden, wird hier jedoch nicht im Detail vorgestellt. Die *draw()*-Funktion

implementiert den Unranking-Algorithmus in den Subklassen. Für die Subklassen bedeutet dies Folgendes: *UrnO* implementiert Algorithmus 2, *UrnOR* Algorithmus 4, *Urn* Algorithmus 7 und *UrnR* Algorithmus 9. Die Algorithmen erhalten nur den Rang *r* als Argument, da sie bereits im Konstruktor mit *n* und *k* initialisiert wurden. Der Rückgabewert ist ein C++ *std::vector* mit der entsprechenden Ziehung. Eine exemplarische Anwendung ist in Abbildung 4 dargestellt.

```
 1. #include "urn.hpp"
 2. #include <iostream>
 3.
 4. int main()
 5. {
 6.     urn::UrnOR urn {3, 3};
 7.
 8.     std::cout << urn::to_string(urn.draw(0)) << std::endl;
 9.     std::cout << urn::to_string(urn.draw(10)) << std::endl;
10.     std::cout << urn::to_string(urn.draw(26)) << std::endl;
11.
12.     //Output
13.     0 0 0
14.     1 0 1
15.     2 2 2
16. }
17.
```

Abbildung 4: Beispiel Anwendung UrnOR

Abbildung 3 zeigt auch, dass der Prototyp leicht erweitert werden kann. Ein Benutzer könnte beispielsweise einen weiteren Typ für klassische Permutationen hinzufügen. Dazu müsste er lediglich eine Implementierung für die *size()*-Funktion und einen Unranking-Algorithmus für die *draw()*-Funktion bereitstellen, beispielsweise den in [Myrvold/Ruskey 2001] beschriebenen Ansatz.

Der Prototyp erfüllt alle definierten Anforderungen und kann im nächsten Schritt durch einen C++-Iterator ergänzt werden.

7.2 Erweiterung des Prototyps um einen C++-Iterator

Kapitel 5 stellt die Standard Template Library und ihre Komponenten vor. Es erklärt, dass der Prototyp einen Container und einen Iterator haben muss, um die Algorithmen der Standardbibliothek nutzen zu können. Die STL bietet verschiedene Container wie Vektoren, Sets oder Arrays zur Verwaltung von Daten (vgl. [Josuttis 2012, 167]). Allen diesen Datenstrukturen ist gemeinsam, dass sie Referenzen auf alle Objekte intern auf ihre eigene Weise verwalten.

Eine Forschungsfrage wirft jedoch Probleme auf. „Inwiefern können kombinatorische Strukturen durch C++-Iteratoren ohne unterliegenden Container umgesetzt werden?“ Dieses

Problem ist durch die Urnenklassen bereits gelöst, da die Ziehungsobjekte bei Bedarf generiert werden. Das bedeutet, dass im Gegensatz zu den STL-Containern keine Notwendigkeit besteht, alle Objekte gleichzeigt zu verwalten. Als Verbindung zwischen Containern und Algorithmen muss im nächsten Schritt eine Iterator Klasse bereitgestellt werden. In diesem Kapitel wird erläutert, wie sich die Verwaltungsbeschränkung auf die Implementierung des Iterators auswirkt. Um die Konformität des Urnen Iterators sicherzustellen, muss dieser schließlich anhand des Konzeptsystems überprüft werden.

Um einen wahlfreien Zugriff auf die einzelnen Ziehungen der Urnentypen zu gewährleisten, wird der Urnen Iterator als ein Random-Access-Iterator implementiert. Das bedeutet, dass er alle Konzepte der Iterator Kategorien erfüllen muss. Um einen Überblick über die Erweiterung des Prototyps zu erhalten, kann das UML-Klassendiagramm in Abbildung 5 verwendet werden.

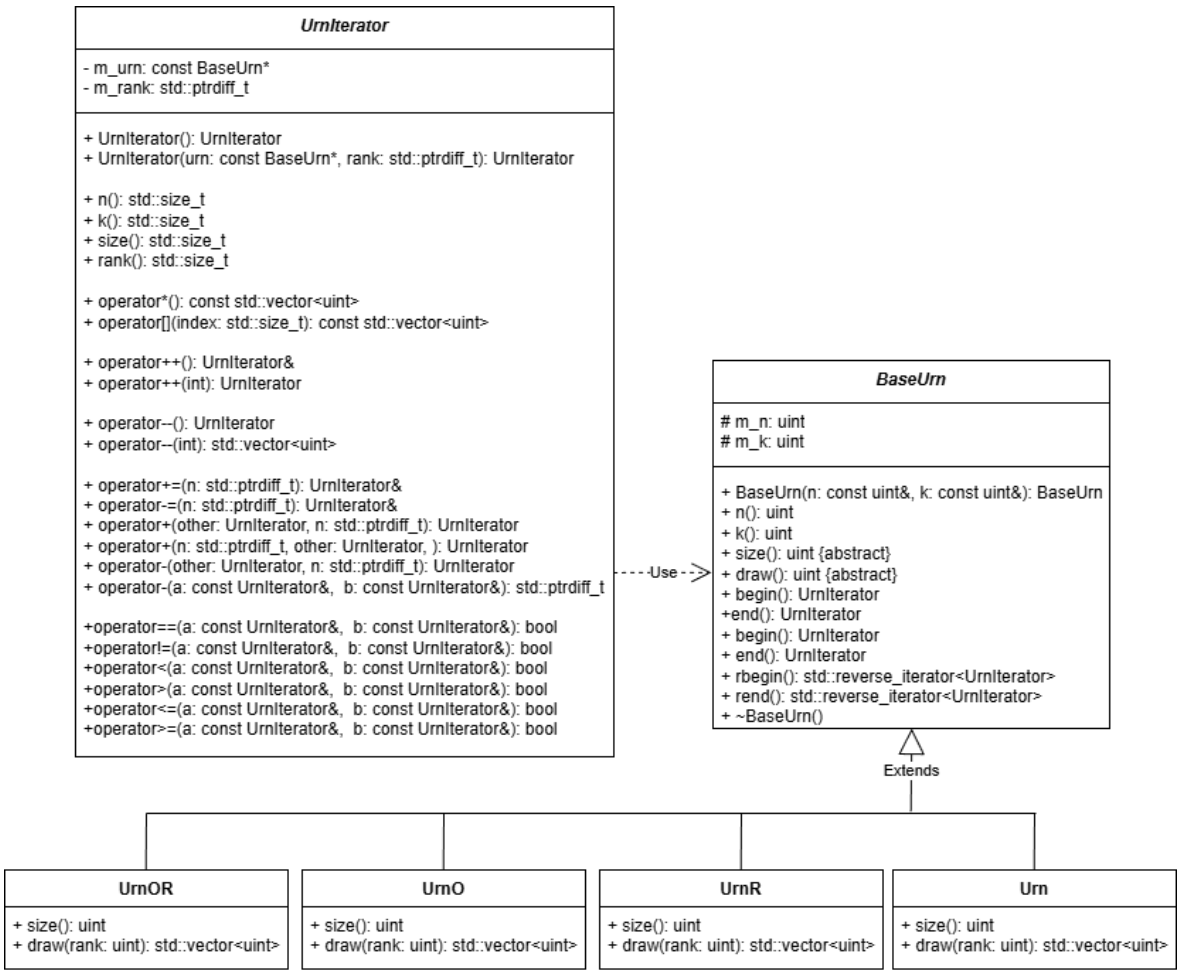

Abbildung 5: UML-Klassendiagramm des Urnenmodell-Prototyps mit Iterator

Abbildung 5 erweitert das UML-Klassendiagramm des Prototyps um einen Random-Access-Iterator. Der ursprüngliche Prototyp muss nur geringfügig in der abstrakten

Basisklasse *BaseUrn* geändert werden. Sie erhält vier neue Memberfunktionen: *begin()*, *end()*, *rbegin()* und *rend()*. Alle Klassen, die mithilfe von Iteratoren durch ihre Elemente navigieren, bieten dieselben Funktionen. Diese dienen als Schnittstelle für den Iterator. Die wichtigsten Mitgliedsfunktionen sind *begin()* und *end()*. Die *begin()*-Funktion gibt einen Iterator zurück, der das Element an der ersten Position repräsentiert. Die *end()*-Funktion hingegen repräsentiert nicht das letzte Element, sondern die Position dahinter (vgl. [Josuttis 2012, 189]). Ähnlich wie diese Funktionen sind *rbegin()* und *rend()*. Sie bieten eine Schnittstelle, mit der Elemente in umgekehrter Reihenfolge durchlaufen werden (vgl. [Josuttis 2012, 214]). Der iterator-Header der Standardbibliothek stellt zu diesem Zweck den Iterator-Adapter *std::reverse_iterator* zur Verfügung (vgl. [cppreference.com 2025e]). Im Fall von *rbegin()* wird dieser mit *begin()* initialisiert und dann zurückgegeben. Dies funktioniert analog für *rend()* und *end()*. Im nächsten Schritt wird der Prototyp um die Klasse *UrnIterator* erweitert.

Um die Verwaltungsrestriktion einzuhalten, muss der Iterator das Verhalten eines Containers simulieren. Zu diesem Zweck erhält der *UrnIterator* das Attribute *m_rank*, das kontinuierlich geändert wird, um einzelne Indexzugriffe zu ermöglichen. Dies kann dann, während der Dereferenzierung verwendet werden, um die gewünschte Ziehung bei Bedarf zu generieren. Darüber hinaus überlädt der Iterator alle für einen Random-Access-Iterator erforderlichen Operatoren. Abbildung 6 zeigt als Beispiel die Implementierung des Präfix-Inkrement Operators.

```
1. UrnIterator& operator++(){
2.     ++m_rank;
3.     return *this;
4. }
5.
```

Abbildung 6: UrnIterator Implementation des Präfix-Increment Operators

Eine vollständige Vorstellung aller Operatoren wird hier ausgelassen. Die Implementierung der anderen Operatoren kann in Anhang A gefunden werden. Im nächsten Schritt kann die Iterator-Klasse mithilfe des Konzeptsystems überprüft werden.

Zur Verifikation von Iteratoren stellt die C++-Standardbibliothek Konzepte im iterator-Header zur Verfügung. Diese werden mit der zu überprüfenden Iterator Klasse initialisiert und können in Verbindung mit einem *static_assert* ausgewertet werden. Das Konzept überprüft, ob alle erforderlichen Anforderungen erfüllt sind, falls nicht, generiert *static_assert* eine Fehlermeldung (vgl. [cppreference.com 2025d]). Es wird empfohlen

einmal selbstständig *static_assert(std::random_access_iterator<urn::UrnIterator>);* auszuprobieren.

Mit der Erfüllung des Konzepts wurde der Prototyp um einen Iterator erweitert. Bei der Verwendung treten jedoch einige Probleme auf. Die Grundidee hinter dem Prototyp basiert auf dem Prozess des Unranking, d.h. der Generierung einer Ziehung nach Bedarf. Als Folge dessen muss der Iterator ohne die Verwaltung aller Objekte gleichzeitig funktionieren, was die Funktionalität des Prototyps leicht einschränkt.

[cppreference.com 2025d] zeigt, dass das *std::random_access_iterator*-Konzept die Definition des Indexoperators *(engl. subscript operator)* [] erfordert. Der Rückgabewert ist normalerweise eine Referenz auf das gezogene Objekt. Seit Einführung des Konzeptsystems mit C++20 wurde die Anforderung, bei Dereferenzierung einen Lvalue zurückzugeben, aufgehoben. Der *UrnIterator* nutzt diese Tatsache, um stattdessen die Ziehung als Wert zurückzugeben. Dies führt zu Einschränkungen bei der Verwendung des Iterators.

```
1. urn::UrnOR u {3, 3};
2. urn::UrnIterator it {u.begin()};
3. std::cout << urn::to_string(it[4]) << std::endl;
4. it[4] = {1, 2, 3};
5.
```

Abbildung 7: Anwendungsbeispiel des Indexoperators

Abbildung 7 zeigt eine exemplarische Anwendung. Die in Zeile 4 gezeigte Zuweisung unter Verwendung des Indexoperator erzeugt einen Fehler zur Kompilierzeit. Das Verhalten wird durch Abbildung 5 begründet, da der Indexoperator die Ziehung als konstanten Wert zurückgibt.

Kapitel 5 stellt die Komponenten der STL vor. Eine der drei Hauptkomponenten sind Algorithmen wie Such- oder Sortieralgorithmen. Die Daten werden vom Container verwaltet und mithilfe von Algorithmen verarbeitet. Einer dieser STL-Algorithmen ist *std::sort*. Er sortiert die Elemente des Containers nach einer gewünschten Reihenfolge. Bei der Anwendung des Algorithmus auf den *UrnenIterator* tritt jedoch ein Fehler auf. Das Verhalten lässt sich dadurch begründen, dass im *UrnIterator* keine Objekte direkt verwaltet werden, sondern diese bei Bedarf durch den Unranking-Algorithmus generiert werden.

Der Prototyp ist daher in seiner Nutzung eingeschränkt, wie die genannten Beispiele bezeugen. Dennoch muss betont werden, dass diese Operationen im Zusammenhang mit dem Prototyp von geringem Nutzen sind. Das Überschreiben einer Ziehung ist für die Urnen

nicht notwendig, da die Ziehungen durch den Unranking-Algorithmus deterministisch bestimmt werden. Ebenso ist eine Sortieroperation nicht sinnvoll, da die Ziehungen bereits in lexikografischer Reihenfolge generiert werden, wodurch die Anwendung eines Sortieralgorithmus überflüssig wird. Zudem würde eine andere Ordnung gegen die in Kapitel 4.3 beschriebene Beschränkungen verstoßen. Trotz der genannten Einschränkungen kann der *UrnIterator* als STL-konform eingestuft werden, da er das *std::random_access_iterator*-Konzept und damit alle notwendigen Anforderungen erfüllt.

Im nächsten Kapitel werden anhand eines Anwendungsbeispiels nützliche Codeausschnitte präsentiert, um die Vorteile des Prototyps zu veranschaulichen.

7.3 Prototyp Anwendungsbeispiel

Kapitel 3.5 beschreibt ein Anwendungsbeispiel für UrnR, das sich mit der prozeduralen Generierung von Spielwelten befasst. Dort wird eine Ziehung aus der Urne als Startwert für einen solchen Generierungsprozess verwendet. Der Startwert kann dann verwendet werden, um eine Vielzahl verschiedener Spielwelten zu generieren. Die Urne enthält eine Menge $B = \{Wüste, Gras, Meer, Jungel, Berge, Schnee\}$ mit den vordefinierten Gebieten, aus denen eine Auswahl getroffen wird. Je nach gewünschten Weltdesign kann es erforderlich sein, die Gebietstypen einzuschränken oder andere Einschränkungen für die Weltgenerierung festzulegen. Eine mögliche Lösung besteht darin, Filter oder Teilmengen der ursprünglichen Menge zu nutzen. Auf diese Weise könnten die gewünschten Bedingungen erfüllt werden. Die C++-Standardbibliothek bietet geeignete Werkzeuge für die Durchführung solcher Operationen.

Die Ranges-Bibliothek erweitert die Algorithmen und Iteratoren der Standardbibliothek. Sie verwendet Views, bei denen es sich um leitgewichtige Objekte handelt. Diese repräsentieren indirekt Ranges, d.h. iterierbare Sequenzen. Die Bibliothek kann dann zum Erstellen und Bearbeiten dieser Ranges verwendet werden (vgl. [cppreference.com 2025b]).

Angenommen, es soll eine Spielwelt generiert werden, die aus neun Teilgebieten besteht, wobei die Gebiete aus Menge B verwendet werden. Eine erste Anforderung könnte sein, dass die Welt nur Berge und schneebedeckte Gebiete enthalten soll. Die Ranges-Bibliothek bietet zu diesem Zweck eine Filterfunktion. In Abbildung 8 wird diese verwendet, um die Teilmenge der Welten zu generieren, die ausschließlich aus Bergen und Schneegebieten bestehen. Dazu wird jede Ziehung darauf überprüft, ob sie andere Elemente als 4 (Berge) oder 5 (Schnee) enthält.

```
1. auto only_mountain_and_snow =
2.     urn | std::views::filter([](const std::vector<uint>& draw) {
3.     return std::ranges::all_of(draw, [](int value) {
4.         return value == 4 || value == 5;
5.     });
6. });
7.
```

Abbildung 8: Anwendungsbeispiel der Ranges Filterfunktion

In einem zweiten Beispiel sollen nur die Startwerte ausgewählt werden, die mindestens ein Meer und maximal zwei Wüstengebiete enthalten. Um dies zu ermöglichen, zählt die *std::ranges::count*-Funktion die Vorkommen der Gebiete in der Ziehung und wertet sie anschließend aus. Der Code ist in Abbildung 9 dargestellt. Beide Beispiele finden sich in Anhang A wieder.

```
 1. urn::UrnR urn {6, 9};
 2. auto constrained_worlds =
 3. urn | std::views::filter([](const std::vector<uint>& draw) {
 4.     const auto sea_count =
 5.         std::ranges::count(draw, 2);
 6.
 7.     const auto desert_count =
 8.         std::ranges::count(draw, 0);
 9.
10.     return sea_count >= 1 && desert_count <= 2;
11. });
12.
```

Abbildung 9: Anwendungsbeispiel 2 der Ranges Filterfunktion

8 Benchmarks

Dieses Kapitel stellt verschiedene Ansätze für kombinatorische Algorithmen vor und vergleicht sie unter identischen Bedingungen anhand von Zeit- und Speicherbenchmarks mit dem Prototyp. Die Szenarien der Zeit- und Speicherbenchmarks werden erläutert und ihre Ergebnisse präsentiert. Ziel dieses Kapitels ist es, die Leistungsfähigkeit der vorgestellten Algorithmen zu demonstrieren.

Die Benchmarks vergleichen je nach Urnentyp drei oder vier Ansätze. Der erste Kandidat ist der Prototyp mit seinem Iterator (Abbildung 5). Kandidat zwei ist ein naiver Ansatz. Er generiert alle möglichen Ziehungen, indem er die Elemente eines *std::vector* inkrementiert. Zusätzlich werden die Funktionen *unsorted()* und *repetitions()* bereitgestellt. Diese überprüfen die Ziehungen auf ihre Ordnung und darauf, ob Wiederholungen enthalten sind. Auf dieser Grundlage kann entschieden werden, ob eine Ziehung im Urnentyp enthalten ist. Eine Beispielanwendung, die alle *k*-Kombinationen ohne Wiederholung generiert, ist in Abbildung 10 dargestellt. Der Quellcode der Funktionen ist in Anhang A zu finden.

```
 1. int n {3};
 2. int k {2};
 3. std::vector<std::vector<uint>> draw (k,0);
 4. do {
 5.     if(!na::unsorted(draw, k) && !na::repetitions(draw, k))
 6.     {
 7.         std::cout << urn::to_string(draw) << std::endl;
 8.     }
 9. } while (na::increment_draw(draw,n,k));
10.
```

Abbildung 10: Anwendungsbeispiel des naiven Ansatzes für Urn

Ein weiterer Ansatz basiert auf den Algorithmen der Discreture-Bibliothek (https://github.com/mraggi/discreture). Discreture ist eine C++14-Bibliothek zum schnellen und einfachen Durchlaufen verschiedener kombinatorische Strukturen wie Permutationen oder Kombinationen. Als letzter Kandidat wird auch die *std::next_permutation()*-Funktion in den Benchmarks für k-Permutation ohne Wiederholung getestet (vgl. [cplusplus.com 2025]).

Um die Vergleichbarkeit der Benchmarks zu sichern, wurden alle Benchmarks auf einem System mit identischer Hard- und Softwarekonfiguration durchgeführt. Das Testsystem lief unter Ubuntu 24.04.3 LTS und war mit einem Intel Core i5-13600KF und 32 GB DDR5-5600 RAM ausgestattet. Alle Programme wurden mit G++ 13.3.0 unter Verwendung des C++20-Standards kompiliert.

8.1 Zeitbenchmarks

Die naheliegendste Methode zur Bestimmung der Leistungsfähigkeit der verschiedenen Ansätze bestehen darin, die Laufzeit zu messen, während die Algorithmen eine Aufgabe bearbeiten. Zu diesem Zweck werden in den Benchmarks zwei verschiedene Szenarien getestet. Das eine ist eine vollständige Iteration aller möglichen Ziehungen, das andere der Zugriff auf eine bestimmte Ziehung. Hierfür generiert ein Pseudozufallszahlengenerator eine Zahl im Bereich zwischen 0 und der jeweiligen *size()*-Funktion minus 1.

Das Catch2-Framework (https://github.com/catchorg/Catch2) wird eingesetzt, um standardisierte und vergleichbare Messungen zu erhalten. Zusätzlich zu seiner Hauptanwendung für Unit-Tests bietet das C++-Framework auch Benchmarking Funktionen. Die Benchmarks verwenden die Version 3.12.0. Die Benchmarks selbst sowie eine Konfigurationsdatei finden sich in Anhang A. Aus Gründen der Reproduzierbarkeit enthält die Konfigurationsdatei Erklärungen und Anweisungen dazu, wie die Benchmarks kompiliert und ausgeführt wurden.

Im vollständigen Iterationsszenario generieren die Kandidaten jede mögliche Permutation bzw. Kombination. Dazu werden die verschiedenen Methoden mit denselben Startwerten *n* und *k* initialisiert. Der Benchmark wiederholt diese Aufgabe mit vier Wertpaaren für jeden Urnentyp. Um repräsentative und statistisch relevante Ergebnisse zu erhalten, wurde ein Stichprobenumfang von 500 gewählt. Tabelle 3 zeigt die Ergebnisse des vollständigen Iterationsszenario für *k*-Permutation mit Wiederholung.

Urnentyp|Algorithmus|n|k	Mittelwert in [µs]	KI min in [µs]	KI max in [µs]	Anzahl der Ziehungen
UrnOR|urn::UrnOR|n=6|k=2	0,727	0,702	0,785	36
UrnOR|na::naive_approach|n=6|k=2	0,035	0,034	0,035	36
UrnOR|discreture::multisets|n=6|k=2	0,052	0,052	0,052	36
UrnOR|urn::UrnOR|n=10|k=3	19,595	19,103	21,043	1000
UrnOR|na::naive_approach|n=10|k=3	0,515	0,512	0,520	1000
UrnOR|discreture::multisets|n=10|k=3	0,659	0,635	0,762	1000
UrnOR|urn::UrnOR|n=15|k=4	1002,100	997,112	1010,460	50625
UrnOR|na::naive_approach|n=15|k=4	24,920	24,660	25,758	50625
UrnOR|discreture::multisets|n=15|k=4	35,840	35,188	37,258	50625
UrnOR|urn::UrnOR|n=20|k=5	66052,200	65996,600	66110,300	3200000
UrnOR|na::naive_approach|n=20|k=5	1857,090	1851,300	1864,150	3200000
UrnOR|discreture::multisets|n=20|k=5	2336,550	2309,400	2366,550	3200000

Tabelle 3: Benchmark: Volle Iteration k-Permutation mit Wiederholung

Tabelle 3 zeigt den Mittelwert für eine Iteration in Mikrosekunden für jeden Kandidaten. Da der Mittelwert allein wenig aussagekräftig ist, wird auch das Konfidenzintervall (KI) angegeben, um die Streuung der Ergebnisse zu berücksichtigen. Als Konfidenzniveau wurde ein Wert von 0,95 festgelegt. Das bedeutet, dass der wahre Mittelwert der Laufzeit mit einer Wahrscheinlichkeit von 95% zwischen dem minimalen und maximalen Konfidenzintervall liegt. Um das Wachstum der Werte besser einordnen zu können, wird in der letzten Spalte die Anzahl der möglichen Ziehungen angegeben. Die Werte wurden anhand der Formeln aus Abbildung 1 berechnet. Die Ergebnisse für die anderen Urnentypen finden sich in Anhang B.

Bevor die Ergebnisse ausgewertet werden können, muss auf einige Schwächen der Benchmarks eingegangen werden. Die C++-Standardbibliothek bietet nur einen kombinatorischen Algorithmus für k-Permutationen ohne Wiederholung und kann daher nicht zur Untersuchung der anderen Urnentypen verwendet werden. Außerdem können bei k-Permutationen ohne Wiederholung nur Wertpaare mit $n = k$ verglichen werden. Diese Einschränkung resultiert aus der Implementierung der Algorithmen *std::next_permutation* und *discreture::permutations*, da diese die Auswahl einer Teilmenge k nicht unterstützen. Die Discreture-Bibliothek bietet keinen direkten Kandidaten für k-Permutationen und k-Kombinationen mit Wiederholung. Für k-Permutationen mit Wiederholung kann die Funktion *discreture::multisets* verwendet werden. Sie erstellt die Ziehungen vollständig, jedoch nicht in lexikografischer Reihenfolge. Um einen Kandidaten für k-Kombinationen mit Wiederholung bereitzustellen, wird die Funktion *discreture::combinations* mit der Vorgehensweise aus Algorithmus 8 verwendet. Nachdem nun die Einschränkungen erläutert wurden, können im nächsten Schritt die Ergebnisse ausgewertet werden.

Bei allen Urnentypen weichen die Konfidenzintervalle nur geringfügig vom Mittelwert ab. Dies deutet auf eine hohe Stabilität der Messergebnisse hin. Der Prototyp weist bei allen Urnentypen eine schlechte Skalierbarkeit auf. Die Ursache dafür ist der Unranking-Algorithmus, da dieser die Ziehung vollständig neu berechnet und nicht auf Zwischenergebnissen aufbauen kann. Der naive Ansatz hat die kürzeste Laufzeit für k-Permutation mit Wiederholung, da er nur das vorherige Zwischenergebnis inkrementieren muss. Bei den anderen Urnentypen weist er jedoch bei großen Werten durchweg die schlechteste Laufzeit auf. Der Grund dafür ist die Anwendung der Filterfunktionen *unsorted()* und *repetitions()*. Im schlimmsten Fall müssen die Funktionen den Vektor vollständig durchlaufen und skalieren schlecht mit steigendem Wert von k, d.h. der Länge des Vektors. Für den Urnentyp UrnO liefert die Standardbibliothek die besten Ergebnisse und übertrifft die anderen Ansätze bei einer großen Anzahl von Ziehungen signifikant. Die Discreture-Bibliothek erweist sich für allgemeine Anwendungsfälle über verschiedene Urnentypen hinweg als die effizienteste.

Im zweiten Szenario generieren die Kandidaten eine zufällig gewählte Ziehung. Hierfür erzeugt ein Pseudozufallszahlengenerator eine Zahl im Bereich zwischen 0 und der jeweiligen *size()*-Funktion minus 1. Die Struktur entspricht im Allgemeinem der des ersten Szenarios. Tabelle 4 zeigt die Ergebnisse für k-Permutation mit Wiederholung

Urnentyp|Algorithmus|n|k	Mittelwert in [µs]	KI min in [µs]	KI max in [µs]	Anzahl der Ziehungen
UrnOR|urn::UrnOR|n=6|k=2	0,013	0,013	0,013	36
UrnOR|na::naive_approach|n=6|k=2	0,027	0,027	0,027	36
UrnOR|discreture::multisets|n=6|k=2	0,057	0,056	0,059	36
UrnOR|urn::UrnOR|n=10|k=3	0,012	0,012	0,013	1000
UrnOR|na::naive_approach|n=10|k=3	0,274	0,270	0,282	1000
UrnOR|discreture::multisets|n=10|k=3	0,071	0,070	0,074	1000
UrnOR|urn::UrnOR|n=15|k=4	0,013	0,013	0,014	50625
UrnOR|na::naive_approach|n=15|k=4	12,882	12,440	13,328	50625
UrnOR|discreture::multisets|n=15|k=4	0,096	0,095	0,097	50625
UrnOR|urn::UrnOR|n=20|k=5	0,014	0,013	0,014	3200000
UrnOR|na::naive_approach|n=20|k=5	819,097	775,765	862,670	3200000
UrnOR|discreture::multisets|n=20|k=5	0,115	0,115	0,116	3200000

Tabelle 4: Benchmark: Spezifische Ziehung k-Permutation mit Wiederholung

Die Ergebnisse der iterativen Ansätze sind am offensichtlichsten. Sowohl der naive Ansatz als auch *std::next_permutation* weisen eine schlechte Skalierbarkeit auf. Darüber hinaus haben diese Ansätze im Vergleich zu den anderen Methoden ein größeres Konfidenzintervall.

Die Ursache hierfür ist die Vorgehensweise, mit dem eine spezifische Ziehung unter Verwendung eines iterativen Ansatzes generiert wird. Dabei müssen zunächst alle vorherigen Ziehungen sequenziell berechnet werden. Der tatsächliche Rechenaufwand hängt daher von der zufällig generierten Zahl ab. Diese Abhängigkeit führt zu einem erhöhten Konfidenzintervall und einer längeren Laufzeit. Im Gegensatz zu iterativen Verfahren haben die anderen Ansätze eine konstante Laufzeit und kleine Konfidenzintervalle für alle Urnentypen. Sowohl der Prototyp als auch die Funktionen der Discreture-Bibliothek können die Ziehung direkt Zufallszahl generieren.

Die Discreture-Bibliothek erweist sich in beiden Szenarien als effiziente Variante. Der Grund dafür ist die Implementierung der einzelnen kombinatorischen Funktionen der Bibliothek. Sie verwendet eine Verbindung aus Unranking-Algorithmen und iterativer Methode. Für die

discreture::permutations wird beispielsweise ein Iterator Objekt erstellt, das je nach Operator die am besten geeignete Variante verwendet. Der Aufruf des Index-Operators würde den Unranking-Algorithmus aufrufen, um eine spezifische Ziehung zu generieren. In Zwischen würde die Anwendung, wie im ersten Szenario, eine Inkrementierungs Funktion aufrufen, die intern die zuletzt generierte Ziehung mithilfe *std::next_permutation* in die nächste Ziehung ändert. Das Iterator Objekt erbt von der Klasse *boost::iterator_facade* der Boost-Bibliothek, die diesen Ansatz optimiert. Eine detaillierte Analyse der Funktionsweise wird nicht bereitgestellt, da dies den Rahmen dieser Bachelorarbeit überschreiten würde. Es wird stattdessen auf den Quellcode (https://github.com/mraggi/discreture) verwiesen.

8.2 Speicherbenchmarks

Als weitere Methode zur Messung der Leistungsfähigkeit der verschiedenen Ansätze werden in diesem Kapitel exemplarische Speicherbenchmarks durchgeführt. Da sich das Verhalten der Ansätze für die einzelnen Urnentypen nicht unterscheidet, werden lediglich beispielhafte Speicherbenchmarks für k-Kombinationen ohne Wiederholung durchgeführt. Die Benchmarks untersuchen die drei Ansätze aus Kapitel 8.1. Die Klasse *Urn* wird als Kandidat für den Prototyp untersucht, und die Funktion *discreture::combinations* für die Discreture-Bibliothek. Der naive Ansatz wird durch die Klasse NaiveUrn erweitert, die alle Ziehungen während der Initialisierung generiert.

Da Catch2 leider keine Speicherbenchmark Funktionalitäten anbietet, wird stattdessen der Massif- Profiler aus dem Valgrind Framework verwendet (https://valgrind.org/docs/manual/ms-manual.html). Die Benchmarks verwenden die Version 3.22.0. Der Profiler misst den vom Programm verwendeten Speicher. Während der Laufzeit des Programmes werden Snapshots erstellt, um den aktuellen Speicherverbrauch zu messen. Der Massif-Visualizer wird ebenfalls verwendet, um die Visualisierung der Daten zu vereinfachen.

Das Szenario der Speicherbenchmarks entspricht im Allgemeinem dem zweiten Zeitbenchmark Szenario. Das heißt es wird aus der Menge der k-Kombination ohne Wiederholung eine zufällige Ziehung generiert. Ziel des Szenarios ist es, den maximalen Speicherverbrauch jedes Ansatzes zu ermitteln. Dazu ist es wichtig, die initiale Erstellung des Kandidaten in das Szenario einzubeziehen. Die verschiedenen Methoden werden mit denselben Startwerten $n = 20$ und $k = 5$ initialisiert. Um repräsentative und statistisch Relevante Ergebnisse zu erhalten, wurde einen Stichprobenumfang von 1000 gewählt. Die Benchmarks finden sich in Anhang A. Aus Gründen der Reproduzierbarkeit enthält die

Konfigurationsdatei Erklärungen und Anweisungen dazu, wie die Benchmarks kompiliert und ausgeführt wurden.

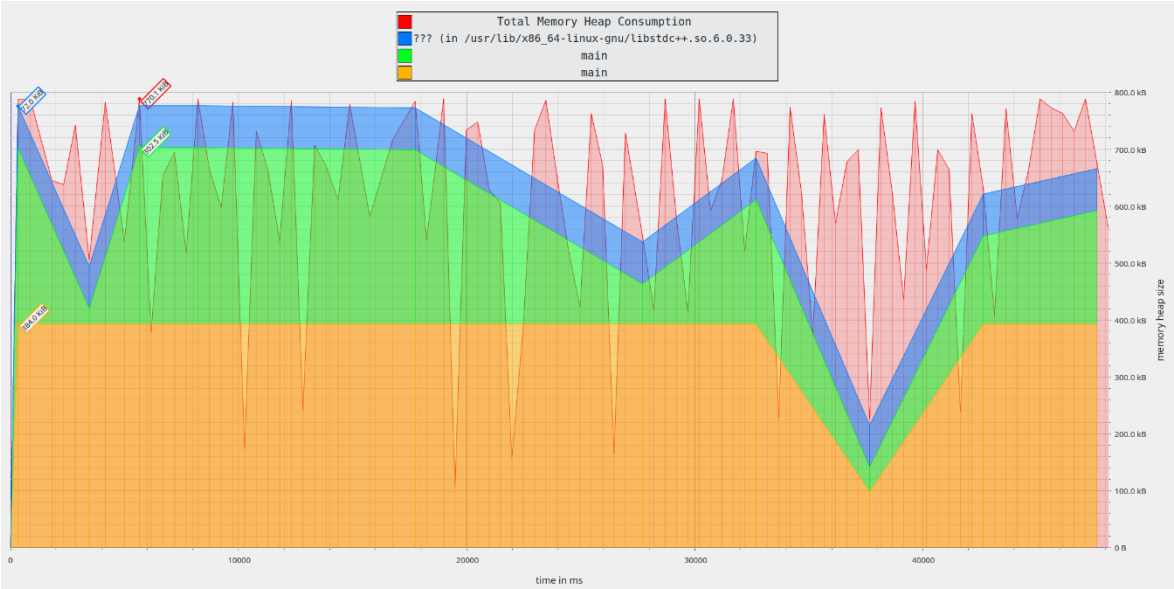

Abbildung 11: Benchmark: Speicherverbrauch des naiven Ansatzes NaiveUrn

Abbildung 11 zeigt die Ergebnisse des naiven Ansatzes *NaiveUrn*. Sie zeigt die aktuelle Speicherauslastung des Speichers in Kilobyte im Verhältnis zur Laufzeit in Millisekunden. Der naive Ansatz erreicht seinen absoluten Höchstwert zu Beginn und hat somit eine Auslastung von 770,1 Kibibyte, was umgerechnet 788,5824 Kilobyte entspricht. Die Schwankungen werden durch die kontinuierliche Neuinitialisierung des *NaiveUrn* Objektes verursacht. Die Klasse *NaiveUrn* generiert alle Ziehungen, wenn der Konstruktor aufgerufen wird. Das bedeutet, dass $K(20,5) = 15504$ Vektoren der Länge 5 generiert und dann gespeichert werden. Der Graph weist nur wenige Schwankungen auf, da während der Laufzeit lediglich 100 Snapshots (Wertpaare) erfasst werden, anstatt der erwarteten 1000 aufgrund des Stichprobenumfangs.

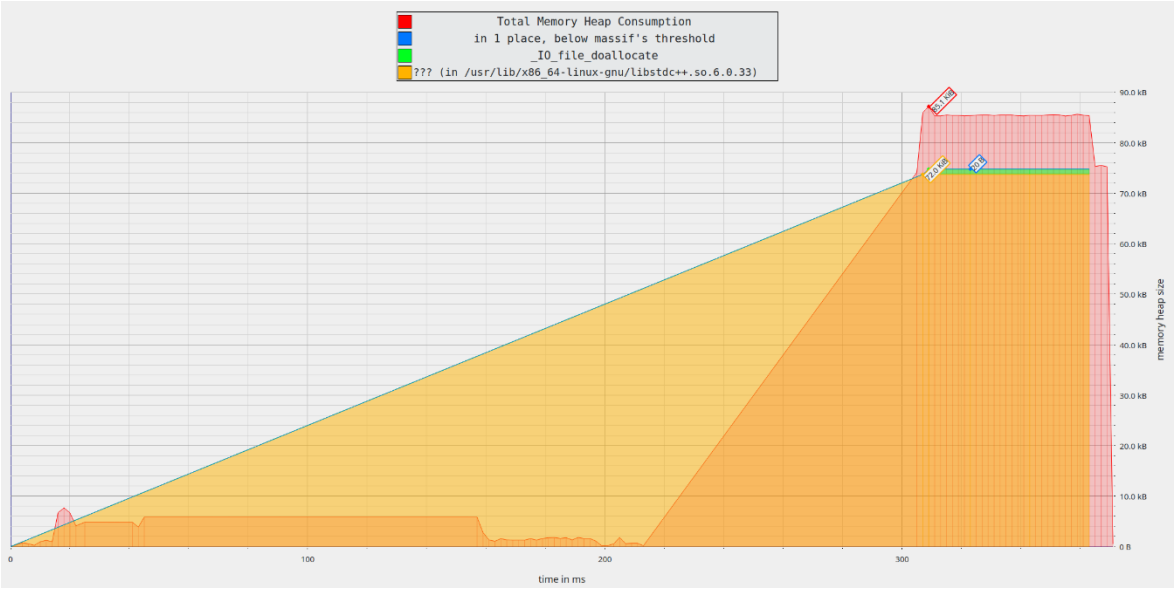

Abbildung 12: Benchmark: Speicherverbrauch des Prototyps Urn

Abbildung 12 zeigt den Speicherverbrauch der Klasse *Urn* des Prototyps. Im Gegensatz zum naiven Ansatz fällt auf, dass die Laufzeit wesentlich kürzer ist. Dieser Unterschied entsteht, weil bei der Neuinitialisierung des *Urn* Objektes die Ziehungen nicht vorab generiert werden. Aufgrund der kürzeren Laufzeit fällt das Gewicht des Initialisierungs- und Verwaltungs-Overhead des Compilers bzw. der Laufzeitumgebung sowie der eingebundenen Bibliotheken umso stärker auf. Der lineare Anstieg bis zum Ende ist fast ausschließlich darauf zurückführen. Die eigentliche Aufgabe des Szenarios, d.h. die kontinuierliche Neuinitialisierung und Generierung der Ziehung, finden im konstanten Abschnitt des Graphen statt. Dabei entstehen, anders als beim naiven Ansatz nur geringe Schwankungen. Dieses Verhalten ist dadurch erklären, dass die Klasse *Urn* die Ziehungen nicht vorab generiert, sondern den Unranking-Algorithmus verwendet. Der Prototyp erreicht seinen absoluten Höchstwert mit einer Auslastung von 85,1 Kibibyte was, umgerechnet 87,1424 Kilobyte entspricht.

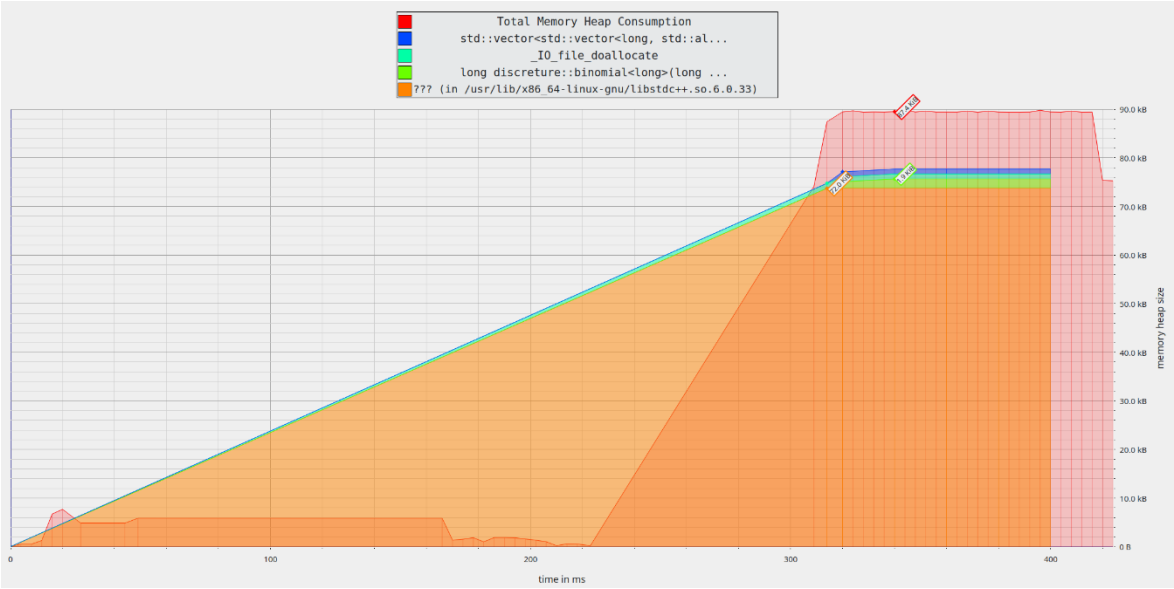

Abbildung 13:Benchmark: Speicherverbrauch Discreture combinations

Abbildung 13 zeigt den Speicherverbrauch der *discreture::combinations* Funktion. Auch hier fällt die kurze Laufzeit im Gegensatz zum naiven Ansatz auf. Der Unterschied ist auch auf den Wegfall vorab generierten Ziehungen zurückzuführen. Auch hier wird der Graph von den Initialisierungs- und Verwaltungs-Overhead sowie der eingebundenen Bibliotheken dominiert. Dieser Ansatz erreicht eine maximale Auslastung von 87,4 Kibibyte, was umgerechnet 89,4976 Kilobyte entspricht.

8.3 Auswertung der Ansätze

Trotz der genannten Einschränkungen liefern die Benchmark Ergebnisse zuverlässige Werte hinsichtlich der Leistungsfähigkeit der einzelnen Ansätze. Der Prototyp ermöglich einen zeit- und speichereffizienten direkten Zugriff auf einzelne Ziehungen für alle Urnentypen. Im vollständigen Iterationsszenario weist er jedoch eine schlechte Skalierbarkeit für größere Werte auf. Die Discreture-Bibliothek liefert im Vergleich zu den anderen Ansätzen sowohl bei den Zeit- als auch bei den Speicherbenchmarks durchweg stabile, effiziente und gut skalierende Ergebnisse. Mit Ausnahme der *k*-Permutation mit Wiederholung weist der naive Ansatz eine deutlich schlechtere Effizienz als die anderen Methoden auf. Dieses Verhalten resultiert aus dem bewusst einfachen und nicht optimierten Design des Ansatzes. Das Ziel der Implementierung ist es, einen Referenzpunkt mit dem ungünstigem Zeit-Speicher-Verhältnis zu zeigen. Für das Szenario des Zugriffs auf eine bestimmte Ziehung wurde die Klasse *NaiveUrn* bewusst nicht verwendet, da ihre Verwendung einem einfachen Indexzugriff entsprechen würde. Im Gegensatz dazu wurde der funktionale Ansatz für die Speicherbenchmarks bewusst abgelehnt. In diesem Fall würde nur der für die Generierung

der Ziehung erforderliche Speicher allokiert werden. In einer Produktivumgebung, in der sowohl Speicherverbrauch als auch die Laufzeit von Bedeutung sind, würden sich beide Varianten als ungeeignet erweisen. Die *NaiveUrn* würde mit unnötigen Ziehungen viel Speicherplatz belegen, während der funktionale Ansatz aufgrund der sequenziellen Generierung und Filterung der Ziehung zu viel Zeit in Anspruch nehmen würde.

Zusammenfassend lässt sich sagen, dass die Discreture-Bibliothek insgesamt die leistungsstärkste Lösung ist. Es ist jedoch zu beachten, dass sie trotz ihrer Leistungsfähigkeit eine Einschränkung aufweist. Im Gegensatz zum Prototyp bietet sie keine Lösungen für *k*-Permutationen mit/ohne Wiederholung sowie *k*-Kombinationen mit Wiederholung. Der Prototyp könnte diese Lücke schließen und ist eine nützliche Ergänzung zur Discreture-Bibliothek. Dazu müsste er, ähnlich wie die Discreture-Funktionen, um einen iterativen Ansatz erweitert werden und je nach verwendetem Operator die effizienteste Vorgehensweise auswählen.

9 Fazit

Nach der Erläuterung der Konzeption und Implementierung des Prototyps sowie der Zeit- und Speicherbenchmarks in den vorangehenden Kapiteln wird in diesem abschließenden Kapitel der Prototyp bewertet und geprüft, ob die gestellten Forschungsfragen beantwortet wurden. Abschließend werden Anregungen zur Erweiterung des Prototyps und Vorschläge für zukünftige Forschungsarbeiten gegeben.

9.1 Bewertung der Ergebnisse der Arbeit

Die Ergebnisse der Arbeit beweisen, dass kombinatorische Strukturen für die vier Grundtypen des Urnenmodells mit Hilfe von Unranking-Algorithmen auch ohne Speicherung von Zwischenergebnissen erzeugt werden können. Der Prototyp macht die Unranking-Algorithmen auf unkomplizierte Weise nutzbar. Das Design der Vererbungshierarchie ist ein intuitives System, das sich für die Erweiterung des Prototyps eignet. Die Erweiterung des Prototyps mit einem C++-Iterator erfüllt zwar die formalen Anforderungen des Konzeptsystems, weist jedoch einige Mängel auf. Die Mängel schränken die Nutzung des Iterators ein, wie Abbildung 7 verdeutlicht. Dennoch muss betont werden, dass diese Operationen im Kontext des Prototyps von geringem Nutzen sind.

In Bezug auf Laufzeit- und Speicherperformance zeigt der Prototyp gemischte Ergebnisse. Der Prototyp ermöglich einen zeit- und speichereffizienten direkten Zugriff auf einzelne Ziehungen für alle Urnentypen. Im vollständigen Iterationsszenario weist er jedoch eine

schlechte Skalierbarkeit für größere Werte auf. Wie in Kapitel 8.3 erwähnt, könnte ein ähnlicher Ansatz wie in der Discreture-Bibliothek verfolgt werden.

Zusammenfassend lässt sich sagen, dass die Generierung kombinatorische Strukturen mithilfe von Unranking-Algorithmen eine speichereffiziente Alternative zu iterativen Ansätzen darstellt. Der entwickelte Prototyp zeigt eine praktische Umsetzung der Konzepte in C++. Dennoch verdeutlichen die Benchmark Ergebnisse, dass der Ansatz nicht für alle Anwendungsszenarien gleichermaßen geeignet ist. Die Arbeit liefert eine fundierte Einordnung der Leistungsfähigkeit des Prototyps und seiner zustandslosen kombinatorischen Algorithmen für die vier Typen des Urnenmodells und kann als Grundlage für weiterführende Forschungsarbeiten dienen.

9.2 Ausblick

Der in der Arbeit entwickelte Prototyp bietet eine solide Grundlage für die speichereffiziente Generierung kombinatorische Strukturen, aber es bleiben weitere Fragen offen, die zu interessanten Ergebnissen führen könnten.

In Kapitel 6 wurde der Umfang der Arbeit auf die vier Grundtypen des Urnenmodells beschränkt. Als weiterführende Literatur wurde hier [Curiel/Genitrini 2024] empfohlen. In dieser Arbeit liefern Curiel und Genitrini Referenzen für die Unranking-Algorithmen des Twelvefold Way. Dazu gehören sowohl die vier Grundtypen des Urnenmodells als auch acht zusätzliche kombinatorische Strukturen. Da sich die Arbeit nur auf Referenzen bezieht, wäre es ein vielversprechender Ansatz, eine umfassende Darstellung aller Unranking-Algorithmen des Twelvefold way zu entwickeln. Eine solche Arbeit könnte nicht nur die theoretischen Ansätze vorstellen, sondern auch konkrete Implementierungen in einer beliebigen Programmiersprache und zusätzlich Vergleich mit Hinsicht auf Laufzeit- und Speicherverhalten präsentieren.

Bei der Suche nach weiteren Ansätzen zur Generierung kombinatorischen Strukturen wurde deutlich, dass sich nur wenige Bibliotheken und Projekte mit diesem Problem befassen. Darüber hinaus decken diese Ansätze nur einen kleinen Teil des Spektrums kombinatorischer Strukturen ab, wie beispielsweise die Discreutre-Bibliothek. Ein weiter Ansatz für zukünftige Arbeiten könnte darin bestehen, den Prototyp auf andere Programmiersprachen zu portieren. In diesem Zusammenhang wäre es sinnvoll, ihn hinsichtlich Laufzeit und Speicherverhalten mit verfügbaren Bibliotheken und Projekten zu vergleichen.

In Kapitel 8.3 wurde darauf hingewiesen, dass die Discreture-Bibliothek Lücken in ihrem Methodenumfang aufweist, von denen einige mit dem Prototyp geschlossen werden könnten. Weitere Arbeit könnte sich auf die Integration oder Weiterentwicklung des Prototyps konzentrieren, um den effizienten Ansatz der Discreture-Bibliothek zu übernehmen.

Ein weniger geeignete, aber dennoch interessante Erweiterungsmöglichkeit für eine wissenschaftliche Arbeit wäre es, den Prototyp, um einige Komfortfunktionen zu ergänzen. Denkbar wäre die Implementierung eines Urnenselektors unter der Verwendung von C++-Templates. Dieser könnte über Argumente den geeigneten Urnentypen auswählen. Darüber hinaus könnte er die Elemente der Permutationen bzw. Kombinationen in benutzerdefinierte Typen oder Strings umwandeln, sodass eine Urne nicht mehr 0,1,2 als Ausgabe zurück gibt, sondern rot, grün, blau.

Als weitere Restriktion der Arbeit wurden Unranking-Algorithmen mit nicht-lexikografischer Ordnung ausgeschlossen. Da die zugrundeliegende Ordnung jedoch nicht für jede Anwendung von relevant ist, könnten Algorithmen mit einer anderen Ordnung zu Performanceverbesserungen führen. Unranking-Algorithmen, die auf anderen Ordnungsregeln basieren, zielen oft darauf ab, eine effizientere Generierung der Ziehungen zu ermöglichen. Vor diesem Hintergrund wäre es interessant, Algorithmen mit anderen Ordnungen, wie z.B. Graycode-Algorithmen, zu untersuchen und sie hinsichtlich ihrer Performance mit lexikografische Ansätze zu vergleichen.

A Quellcode

Die Quellcode-Dateien für den Prototyp sowie die Beispiele aus Kapitel 7.3 und die Benchmarks finden sich auf Github unter https://github.com/Ant2910/combinatorial-unranking.

B Benchmark Ergebnisse

Urnentyp|Algorithmus|n|k	Mittelwert in [µs]	KI min in [µs]	KI max in [µs]	Anzahl der Ziehungen
UrnO|urn::UrnO|n=3|k=3	0,532	0,529	0,538	6
UrnO|na::naive_approach|n=3|k=3	0,086	0,086	0,087	6
UrnO|discreture::permutations|n=3|k=3	0,023	0,021	0,030	6
UrnO|std::next_permutation|n=3|k=3	0,020	0,020	0,020	6
UrnO|urn::UrnO|n=5|k=5	15,900	15,750	16,122	120
UrnO|na::naive_approach|n=5|k=5	11,099	10,897	11,760	120
UrnO|discreture::permutations|n=5|k=5	0,365	0,365	0,367	120
UrnO|std::next_permutation|n=5|k=5	0,208	0,207	0,209	120
UrnO|urn::UrnO|n=7|k=7	747,494	743,182	754,386	5040
UrnO|na::naive_approach|n=7|k=7	3625,040	3614,030	3643,020	5040
UrnO|discreture::permutations|n=7|k=7	14,687	14,640	14,793	5040
UrnO|std::next_permutation|n=7|k=7	8,783	8,598	9,577	5040
UrnO|urn::UrnO|n=8|k=8	6192,720	6170,160	6228,640	40320
UrnO|na::naive_approach|n=8|k=8	73213,400	73064,100	73383,400	40320
UrnO|discreture::permutations|n=8|k=8	119,541	118,565	121,954	40320
UrnO|std::next_permutation|n=8|k=8	68,933	68,593	69,453	40320

Tabelle 5: Benchmark: Volle Iteration k-Permutation ohne Wiederholung

Urnentyp|Algorithmus|n|k	Mittelwert in [µs]	KI min in [µs]	KI max in [µs]	Anzahl der Ziehungen
UrnO|urn::UrnO|n=3|k=3	0,082	0,081	0,083	6
UrnO|na::naive_approach|n=3|k=3	0,052	0,051	0,056	6
UrnO|discreture::permutations|n=3|k=3	0,023	0,022	0,025	6
UrnO|std::next_permutation|n=3|k=3	0,023	0,022	0,024	6
UrnO|urn::UrnO|n=5|k=5	0,127	0,124	0,135	120
UrnO|na::naive_approach|n=5|k=5	4,586	4,454	4,722	120
UrnO|discreture::permutations|n=5|k=5	0,042	0,041	0,045	120
UrnO|std::next_permutation|n=5|k=5	0,125	0,122	0,135	120
UrnO|urn::UrnO|n=7|k=7	0,153	0,146	0,178	5040
UrnO|na::naive_approach|n=7|k=7	1615,500	1530,020	1701,730	5040
UrnO|discreture::permutations|n=7|k=7	0,066	0,065	0,070	5040
UrnO|std::next_permutation|n=7|k=7	5,399	4,498	9,612	5040
UrnO|urn::UrnO|n=8|k=8	0,160	0,158	0,163	40320
UrnO|na::naive_approach|n=8|k=8	33916,100	32163,000	35672,600	40320
UrnO|discreture::permutations|n=8|k=8	0,080	0,079	0,082	40320
UrnO|std::next_permutation|n=8|k=8	38,522	36,687	40,358	40320

Tabelle 6: Benchmark: Spezifische Ziehung k-Permutation ohne Wiederholung

Urnentyp|Algorithmus|n|k	Mittelwert in [µs]	KI min in [µs]	KI max in [µs]	Anzahl der Ziehungen
UrnR|urn::UrnR|n=5|k=2	0,242	0,241	0,244	15
UrnR|na::naive_approach|n=5|k=2	0,042	0,042	0,042	15
UrnR|discreture::combinations|n=5|k=2	0,186	0,185	0,189	15
UrnR|urn::UrnR|n=10|k=3	6,509	6,486	6,549	220
UrnR|na::naive_approach|n=10|k=3	1,719	1,709	1,736	220
UrnR|discreture::combinations|n=10|k=3	2,613	2,600	2,634	220
UrnR|urn::UrnR|n=20|k=5	2998,570	2992,540	3005,900	42504
UrnR|na::naive_approach|n=20|k=5	5495,010	5485,760	5508,680	42504
UrnR|discreture::combinations|n=20|k=5	517,579	515,441	523,619	42504
UrnR|urn::UrnR|n=30|k=6	168805,000	168712,000	168900,000	1623160
UrnR|na::naive_approach|n=30|k=6	1115730,000	1115190,000	1116270,000	1623160
UrnR|discreture::combinations|n=30|k=6	19546,300	19526,800	19572,000	1623160

Tabelle 7: Benchmark: Volle Iteration k-Kombination mit Wiederholung

Urnentyp|Algorithmus|n|k	Mittelwert in [µs]	KI min in [µs]	KI max in [µs]	Anzahl der Ziehungen
UrnR|urn::UrnR|n=5|k=2	0,028	0,027	0,030	15
UrnR|na::naive_approach|n=5|k=2	0,034	0,034	0,035	15
UrnR|discreture::combinations|n=5|k=2	0,038	0,038	0,038	15
UrnR|urn::UrnR|n=10|k=3	0,051	0,049	0,056	220
UrnR|na::naive_approach|n=10|k=3	0,647	0,620	0,707	220
UrnR|discreture::combinations|n=10|k=3	0,057	0,056	0,059	220
UrnR|urn::UrnR|n=20|k=5	0,097	0,097	0,099	42504
UrnR|na::naive_approach|n=20|k=5	1109,340	1027,510	1197,220	42504
UrnR|discreture::combinations|n=20|k=5	0,112	0,107	0,128	42504
UrnR|urn::UrnR|n=30|k=6	0,135	0,134	0,135	1623160
UrnR|na::naive_approach|n=30|k=6	174141,000	160970,000	188367,000	1623160
UrnR|discreture::combinations|n=30|k=6	0,145	0,139	0,170	1623160

Tabelle 8: Benchmark: Spezifische Ziehung k-Kombination mit Wiederholung

Urnentyp|Algorithmus|n|k	Mittelwert in [µs]	KI min in [µs]	KI max in [µs]	Anzahl der Ziehungen
Urn|urn::Urn|n=5|k=2	0,150	0,150	0,153	10
Urn|na::naive_approach|n=5|k=2	0,061	0,061	0,062	10
Urn|discreture::combinations|n=5|k=2	0,021	0,021	0,022	10
Urn|urn::Urn|n=10|k=3	3,460	3,344	3,728	120
Urn|na::naive_approach|n=10|k=3	2,157	2,152	2,166	120
Urn|discreture::combinations|n=10|k=3	0,110	0,109	0,117	120
Urn|urn::Urn|n=20|k=5	981,549	979,513	984,132	15504
Urn|na::naive_approach|n=20|k=5	5616,260	5608,670	5625,080	15504
Urn|discreture::combinations|n=20|k=5	12,954	12,876	13,073	15504
Urn|urn::Urn|n=30|k=6	55049,400	54948,900	55169,900	593775
Urn|na::naive_approach|n=30|k=6	1121360,000	1120770,000	1121990,000	593775
Urn|discreture::combinations|n=30|k=6	472,657	470,930	474,997	593775

Tabelle 9: Benchmark: Volle Iteration k-Kombination ohne Wiederholung

Urnentyp|Algorithmus|n|k	Mittelwert in [µs]	KI min in [µs]	KI max in [µs]	Anzahl der Ziehungen
Urn|urn::Urn|n=5|k=2	0,026	0,025	0,027	10
Urn|na::naive_approach|n=5|k=2	0,035	0,035	0,035	10
Urn|discreture::combinations|n=5|k=2	0,039	0,037	0,043	10
Urn|urn::Urn|n=10|k=3	0,048	0,046	0,056	120
Urn|na::naive_approach|n=10|k=3	0,662	0,652	0,674	120
Urn|discreture::combinations|n=10|k=3	0,055	0,054	0,055	120
Urn|urn::Urn|n=20|k=5	0,091	0,089	0,100	15504
Urn|na::naive_approach|n=20|k=5	946,846	877,867	1021,550	15504
Urn|discreture::combinations|n=20|k=5	0,099	0,098	0,100	15504
Urn|urn::Urn|n=30|k=6	0,123	0,123	0,125	593775
Urn|na::naive_approach|n=30|k=6	149292,000	137641,000	161830,000	593775
Urn|discreture::combinations|n=30|k=6	0,138	0,132	0,164	593775

Tabelle 10: Benchmark: Spezifische Ziehung k-Kombination ohne Wiederholung:

Literaturverzeichnis

Bender, E.A., S. Gill Williamson, Foundations of combinatorics with applications, Dover Publications, Mineola, NY, 2006.
Berger, P., Kombinatorik, Springer Berlin Heidelberg, Berlin, Heidelberg, 2023.
Berger, S., Kravtsiv, A., Schneider, G., Jordan, D., Teaching Ordinal Patterns to a Computer: Efficient Encoding Algorithms Based on the Lehmer Code, in: Entropy, 21 (2019) 10, S. 1023.
Borysenko, O., Horiachev, O., Matsenko, S., Novhorodtsev, A., Berezhna, O., Bobrovs, V., Salgals, T., Porins, J., Factorial Numbers and Their Practical Applications, in: Applied Sciences, 14 (2024) 19, S. 8588.
cplusplus.com, std::next_permutation, 2025, URL: https://cplusplus.com/reference/algorithm/next_permutation/, gelesen am 08.12.2025.
cppreference.com, Iterator library, 2025a, URL: https://en.cppreference.com/w/cpp/iterator.html, gelesen am 13.01.2026.
cppreference.com, Ranges library, 2025b, URL: https://en.cppreference.com/w/cpp/ranges.html, gelesen am 22.01.2026.
cppreference.com, std::pow, std::powf, std::powl, 2025c, URL: https://en.cppreference.com/w/cpp/numeric/math/pow, gelesen am 12.01.2026.
cppreference.com, std::random_access_iterator, 2025d, URL: https://en.cppreference.com/w/cpp/iterator/random_access_iterator.html, gelesen am 14.01.2026.
cppreference.com, std::reverse_iterator, 2025e, URL: https://en.cppreference.com/w/cpp/iterator/reverse_iterator.html, gelesen am 21.01.2026.
Curiel, A., Genitrini, A., Lexicographic Unranking Algorithms for the Twelvefold Way, in: LIPIcs, Volume 302, AofA 2024, 302 (2024).
Frank Ruskey, Combinatorial Generation, Working Version (1j-CSC 425/520). Aufl., 2003.
Genitrini, A., Pépin, M., Lexicographic Unranking of Combinations Revisited, in: Algorithms, 14 (2021) 3, S. 97.
Graham, R.L., Knuth, D.E., Patashnik, O., Concrete mathematics, 2. edition. Aufl., Addison-Wesley, Reading, Mass., 1994.
Höfler, M., Methods for intraday scheduling in particle therapy, 2018.
Josuttis, N.M., The C standard library, 2. ed. Aufl., Addison-Wesley, Upper Saddle River, NJ, Munich, 2012.
Knuth, D.E., The art of computer programming, 2. Aufl., Addison-Wesley, Reading, Mass, 1997.
Kreher, D.L., Stinson, D.R., Combinatorial algorithms, CRC Press, Boca Raton, 1999.

Laisant, C.-A., Sur la numération factorielle, application aux permutations, in: Bulletin de la Société mathématique de France, 2 (1888), S. 176–183.
Mifsud, C.J., Algorithm 154: combination in lexicographical order, in: Communications of the ACM, 6 (1963) 3, S. 103.
Myrvold, W., Ruskey, F., Ranking and unranking permutations in linear time, in: Information Processing Letters, 79 (2001) 6, S. 281–284.
Rosen, K.H., Discrete mathematics and its applications, 7th ed. Aufl., McGraw-Hill, New York, 2012.
Wilde, T., Hess, T., Methodenspektrum der Wirtschaftsinformatik: Überblick und Portfoliobildung (2006).
Witt, K.-U., Elementare Kombinatorik für die Informatik, Springer Fachmedien Wiesbaden, Wiesbaden, 2013.

10 Ehrenwörtliche Erklärung

Ich versichere, dass ich die Bachelorarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Zur Recherche von Quellen und zur Strukturierung der Bachelorarbeit wurde künstliche Intelligenz in Form von ChatGPT eingesetzt. Um grammatikalische und orthografische Fehler zu vermeiden, wurden außerdem KI-gestützte Schreibassistenten wie DeepL und QuillBot verwendet.

Leipzig, 12.02.2026